

REPUBLIQUE DU CAMEROUN

Paix – Travail – Patrie

Etablissement Inter – Etats
d'Enseignement Supérieur

Représentation du Cameroun

IAI CENTRE D'EXCELLENCE
TECHNOLOGIQUE PAUL BIYA

BP 13719 Yaoundé (Cameroun)

Tél. (237) 242 72 99 57

Site web : www.iaicameroun.com

Email : contact@iaicameroun.com



REPUBLIC OF CAMEROUN

Peace – Work – Fatherland

Inter – State Establishment of Higher Education

Representation of Cameroon

IAI PAUL BIYA CENTER
TECHNOLOGICAL EXCELLENCE

PO 41465 Yaoundé (Cameroon)

Phone (237) 242 72 99 57

eb Site : www.iaicameroun.com

Email : contact@iaicameroun.com

EXPOSE D'ORACLE

THEME:

ADMINISTRATION D'UNE BASE DE DONNEES DE GESTION DES FINANCES

THEME:6

Rédigés et présentés par :

- MBANDJO Eugene Junior
- KENSO Sahleh Simon Malley
- ANDEME ABESSOLO Rosy
- NITCHEU Marcel Kevin
- MEDOULOU Noel Junior

CLASSE : GL3C

Sous la supervision de :

M.MEMBOU WILSON

Année académique 2025/2026

SOMMAIRE

SOMMAIRE.....	1
LISTE DES FIGURES	2
RESUME	3
ABSTRACTE.....	4
INTRODUCTION GENERALE	5
PARTIE : PHASE TECHNIQUE.....	6
Dossier 1 : L'Existant.....	7
Dossier 2 : Cahier des charges.....	13
I. Contexte et justification de l'étude / du projet	14
II. Les objectifs de l'étude / du projet.....	16
III. Expression des besoins de l'utilisateur	17
Dossier 3 : analyse	20
Dossier 5 : Dossier de Réalisation :	30
Dossier 6 : Test de fonctionnalités.....	32
Dossier 7 : Guide d'installation et guide d'utilisateur	36
I. CAS PRATIQUE	45
CONCLUSION	77

LISTE DES FIGURES

Figure 1:Formalisme du diagramme de classe.....	28
Figure 2:diagramme de classe du système.....	29
Figure 3:Connexion.....	46
Figure 4:implemттation.....	47
Figure 5:implementation.....	48
Figure 6:implemттation.....	49
Figure 7:connection admin.....	50
Figure 8:creation de la table user.....	51
Figure 9:creation de la table categorie.....	52
Figure 10:TABLE COMPTE.....	53
Figure 11:table transaction_finiance.....	55
Figure 12:table audit_log.....	56
Figure 13:creation des sequence.....	57
Figure 14:insertion de données.....	58
Figure 15:insertion compte.....	59
Figure 16:inertion donnees.....	60
Figure 17;affiche la utilisateur.....	61
Figure 18:affiche un compte.....	61
Figure 19:affiche de la transaction.....	61
Figure 20: insertion des données de l'audit.....	62

RESUME

La gestion efficace des finances constitue un enjeu majeur pour les organisations modernes, qu'elles soient publiques ou privées. Face à l'augmentation du volume des données financières et aux exigences accrues en matière de sécurité, de fiabilité et de traçabilité, l'utilisation d'un système de gestion de base de données performant devient indispensable. Dans ce contexte, ce projet porte sur l'administration d'une base de données de gestion des finances basée sur Oracle. L'objectif principal de ce travail est de concevoir, analyser et administrer une base de données financière capable d'assurer la gestion des recettes, des dépenses, des budgets et des transactions financières, tout en garantissant la sécurité et l'intégrité des données. Pour atteindre cet objectif, une analyse approfondie des besoins a été réalisée à l'aide du langage de modélisation UML, permettant la conception des diagrammes de cas d'utilisation et du diagramme de classes, éléments essentiels pour structurer le système. La base de données a ensuite été mise en œuvre sous Oracle Data base, avec une attention particulière portée à l'administration des utilisateurs, des rôles, des privilèges, des tablespaces, ainsi qu'aux mécanismes de sauvegarde, de restauration et d'optimisation des performances. Des scripts SQL ont été développés afin de créer les tables, définir les contraintes d'intégrité et assurer le bon fonctionnement du système. Ce projet met en évidence l'importance du rôle de l'administrateur de base de données dans la gestion sécurisée des informations financières et démontre qu'Oracle constitue une solution robuste et fiable pour les systèmes financiers modernes.

Mots-clés : Base de données, Oracle, Administration, Gestion financière, UML, Sécurité des données.

ABSTRACTE

Efficient financial management represents a major challenge for modern organizations, whether public or private. With the continuous growth of financial data and increasing requirements in terms of security, reliability, and traceability, the use of a powerful database management system has become essential. In this context, this project focuses on the administration of a financial management database using Oracle. The main objective of this work is to design, analyze, and administer a financial database capable of managing revenues, expenses, budgets, and financial transactions while ensuring data security and integrity. To achieve this goal, a detailed requirements analysis was conducted using the UML modeling language, leading to the creation of use case diagrams and a class diagram, which form the structural foundation of the system. The database was implemented using Oracle Database, with particular emphasis on user management, roles and privileges, tablespaces, as well as backup, recovery, and performance optimization mechanisms. SQL scripts were developed to create tables, define integrity constraints, and ensure the proper functioning of the system. This project highlights the crucial role of database administration in securing financial information and demonstrates that Oracle is a robust and reliable solution for modern financial management systems.

Keywords: Database, **Oracle**, **Administration**, **Financial Management**, **UML**, **Data Security**.

INTRODUCTION GENERALE

Au Cameroun, la gestion des finances au sein des entreprises, des administrations publiques et des organisations privées demeure un enjeu majeur. Les pratiques encore largement basées sur des traitements manuels ou des systèmes informatisés peu sécurisés entraînent de nombreuses difficultés telles que les erreurs de saisie, la perte d'informations, le manque de traçabilité des transactions financières ainsi que les risques de fraude. Ces insuffisances compromettent la fiabilité des données financières et la prise de décisions stratégiques. Face à ces défis, il devient indispensable de mettre en place des solutions numériques modernes, performantes et sécurisées, capables d'assurer une gestion efficace et transparente des ressources financières. C'est dans ce contexte qu'a été conçu le présent projet portant sur l'administration d'une base de données de gestion des finances sous Oracle. Le principal enjeu de ce travail est de concevoir et d'administrer une solution informatique fiable permettant de centraliser, sécuriser et gérer efficacement les données financières d'une organisation. L'utilisation du système de gestion de base de données Oracle Database s'inscrit dans cette logique, en raison de ses fonctionnalités avancées en matière de sécurité, de gestion des transactions et de performance. L'objectif général de ce projet est de mettre en place une base de données financière administrée de manière optimale, garantissant l'intégrité, la confidentialité et la disponibilité des informations. Plus spécifiquement, il vise à structurer les données financières relatives aux recettes, aux dépenses, aux budgets et aux transactions, à assurer une gestion rigoureuse des utilisateurs et de leurs privilèges, à mettre en œuvre des mécanismes de sauvegarde et de restauration des données, ainsi qu'à optimiser les performances du système. Le projet intègre également une phase d'analyse et de conception basée sur le langage de modélisation UML, notamment à travers la réalisation d'un diagramme de classes permettant de représenter clairement les différentes entités du système et leurs relations. Ce rapport présente dans un premier temps les étapes de conception à travers les méthodes d'analyse UML. La troisième partie est consacrée au choix des technologies et à l'implémentation du projet. Enfin, une dernière section présente les résultats obtenus

PARTIE : PHASE TECHNIQUE

Il a été question pour nous dans la phase d'insertion de nous familiariser avec notre structure d'accueil. L'accent sera mis dans cette phase sur les différentes caractéristiques, spécificités et attentes du sujet soumis à notre étude.

- **DOSSIER 1 : L'EXISTANT**
- **DOSSIER 2 : L'EXISTANT**
- **DOSSIER 3 : ANALYSE**
- **DOSSIER 4 : CONCEPTION**
- **DOSSIER 5 : REALISATION**
- **DOSSIER 6 : TESTS**
- **DOSSIER 7 : GUIDE D'INSTALLATION**

Dossier 1 : L'Existant

Résumé :

Le dossier de l'existant analyse en détail le système actuel, décrivant ses fonctionnalités, ses performances et ses limitations. Il identifie les besoins non satisfaits et les problèmes à résoudre, fournissant ainsi une base solide pour guider le développement de la nouvelle application. Ce dossier permet de comprendre les forces et faiblesses du système en place et d'établir des recommandations pour améliorer et optimiser la solution future.

Aperçu :

- I. PRESENTATION DU THEME**
- II. ETUDE L'EXISTANT**
- III. CRITQUE DE L'EXISTANT**
- IV. PROBLEMATIQUE**
- V. PROPOSITION DE LA SOLUTION**

I. Présentation du thème / projet

Le thème de notre rapport s'intitule : **Administration d'une base de données de gestion des finances sous Oracle**. La gestion financière constitue un pilier essentiel du bon fonctionnement des entreprises, des administrations publiques et des organisations privées. Elle repose sur le suivi rigoureux des recettes, des dépenses, des budgets et des transactions financières, afin de garantir la transparence, la fiabilité des informations et une prise de décision efficace. Le présent projet porte sur la mise en place et l'administration d'une base de données de gestion des finances, conçue pour centraliser et sécuriser l'ensemble des données financières d'une organisation. La solution proposée s'appuie sur le système de gestion de base de données Oracle, reconnu pour sa robustesse, ses mécanismes avancés de sécurité et ses performances élevées.

La première version du système de gestion financière intègre les fonctionnalités suivantes :

- Inscription et authentification des utilisateurs (administrateur, comptable, responsable financier).
- Gestion des comptes financiers.
- Enregistrement des recettes et des dépenses.
- Gestion et suivi des budgets.
- Historisation et traçabilité des transactions financières.
- Gestion des rôles et des privilèges utilisateurs.
- Sécurisation des données financières par des mécanismes d'accès contrôlés.
- Sauvegarde et restauration des données.
- Génération de rapports financiers pour l'aide à la décision.

Cette solution vise à répondre aux besoins des organisations en matière de gestion financière en apportant :

- Une meilleure organisation et centralisation des données financières.
- Une réduction des erreurs liées aux traitements manuels.
- Une amélioration de la sécurité et de la confidentialité des informations.

- Une meilleure traçabilité et fiabilité des opérations financières.

II. Étude de l'existant

Tout projet informatique, pour être pertinent et efficace, nécessite une compréhension approfondie du système existant. Il est essentiel d'analyser les méthodes actuellement utilisées pour la gestion des finances afin d'identifier leurs forces, leurs faiblesses et leurs limites. Dans le cadre de ce projet, nous avons étudié les pratiques de gestion financière en vigueur dans plusieurs structures locales, en particulier celles qui reposent encore sur des outils peu automatisés ou insuffisamment sécurisés.

Actuellement, dans de nombreuses organisations camerounaises, la gestion des finances est assurée à l'aide de registres manuels, de fichiers Excel ou de logiciels peu adaptés aux exigences de sécurité et de performance. Ces méthodes permettent un suivi basique des opérations financières, mais présentent de nombreuses insuffisances dès que le volume des données augmente.

L'utilisation de fichiers Excel, bien que répandue, pose plusieurs problèmes, notamment la duplication des données, l'absence de contrôle d'accès rigoureux, la difficulté de gestion multi-utilisateurs et le risque élevé de perte ou de modification non autorisée des informations. Par ailleurs, les sauvegardes ne sont pas systématiques, ce qui expose les organisations à des pertes de données critiques en cas de panne ou de mauvaise manipulation.

Dans certains cas, des solutions informatiques existent mais elles sont souvent mal administrées, avec une gestion approximative des utilisateurs, des privilèges et des mécanismes de sécurité. L'absence d'une véritable politique d'administration de base de données entraîne des failles de sécurité, une faible performance du système et un manque de fiabilité des données financières.

Ainsi, malgré l'existence de quelques outils numériques, la majorité des systèmes actuels ne garantissent ni l'intégrité, ni la confidentialité, ni la disponibilité des données financières, ce qui constitue un frein majeur à une gestion financière moderne et efficace.

III. Critique de l'existant

L'analyse de l'existant nous a permis de dégager les critiques suivantes, résumées dans le tableau ci-dessous :

Tableau : Critique de l'existant

Critiques	Limites	Solutions
Gestion manuelle des finances	Risque élevé d'erreurs, lenteur, absence de traçabilité	Mise en place d'une base de données centralisée
Utilisation de fichiers Excel	Duplication des données, sécurité faible, accès non contrôlé	Migration vers un SGBD robuste (Oracle)
Absence de gestion des rôles	Accès non autorisé aux données sensibles	Gestion des utilisateurs, rôles et privilèges
Faible sécurité des données	Risques de fraude et de modification non autorisée	Sécurisation via authentification et autorisations
Absence de sauvegarde automatique	Perte de données en cas de panne	Mise en place de mécanismes de backup et recovery
Difficulté d'analyse financière	Données dispersées et incohérentes	Centralisation et génération de rapports financiers

Faible performance des systèmes existants	Temps de réponse lent	Optimisation des performances avec Oracle
--	-----------------------	---

Tableau 1;critique existant

IV. Problématique

La gestion financière au sein des organisations camerounaises est confrontée à de nombreuses difficultés liées à l'utilisation de systèmes peu sécurisés, mal structurés et insuffisamment administrés. Les méthodes actuelles ne garantissent pas une gestion fiable des données financières, ce qui peut entraîner des erreurs de calcul, des pertes d'informations et des décisions basées sur des données incorrectes.

Face à ces contraintes, il devient nécessaire de concevoir une solution informatique capable de :

- Centraliser les données financières,
- Garantir leur intégrité et leur sécurité,
- Assurer une gestion rigoureuse des utilisateurs et des accès,
- Améliorer la fiabilité et la performance du système.

La problématique peut donc être formulée ainsi :

Comment assurer une administration efficace, sécurisée et fiable des données financières au sein d'une organisation à l'aide d'un système de gestion de base de données Oracle ?

V. Proposition de solution

Pour répondre aux limites identifiées, il est proposé de mettre en place une base de données de gestion des finances administrée sous Oracle. Cette solution vise à moderniser et sécuriser la gestion financière en offrant un système centralisé, fiable et performant.

La solution permettra de structurer les données financières relatives aux comptes, aux recettes, aux dépenses, aux budgets et aux transactions. Elle intégrera une gestion stricte des utilisateurs, des rôles et des privilèges afin de garantir la confidentialité des informations sensibles. Des mécanismes de sauvegarde et de restauration seront également mis en place pour assurer la disponibilité des données en toutes circonstances.

Grâce à Oracle, la base de données offrira une meilleure performance, une sécurité renforcée et une administration efficace, facilitant ainsi le suivi financier et l'aide à la prise de décision. Cette solution est évolutive et pourra, à terme, intégrer des fonctionnalités supplémentaires telles que des tableaux de bord avancés, l'automatisation des rapports financiers et l'interconnexion avec d'autres systèmes d'information.

Dossier 2 : Cahier des charges

Résumé :

Le cahier des charges est un document contractuel établi entre le maître d'œuvre et le maître d'ouvrage qui étale les besoins du client. Il étudie et présente avec exactitude les exigences formulées par les utilisateurs en ce qui concerne le projet, son déroulement et les résultats.

Aperçu :

- I. Contexte et justification**
- II. Objectif d'étude**
- III. Expression des besoins**

I. Contexte et justification de l'étude / du projet

1. Contexte

La gestion des finances constitue un enjeu stratégique pour les organisations publiques et privées au Cameroun. Avec l'évolution des activités économiques, l'augmentation du volume des transactions et la nécessité de produire des informations financières fiables, les structures sont de plus en plus confrontées à des défis liés à l'organisation, à la sécurité et à la fiabilité des données financières.

Dans de nombreuses institutions camerounaises, la gestion financière repose encore sur des méthodes traditionnelles ou semi-informatisées telles que les registres manuels, les fichiers Excel ou des logiciels non intégrés. Ces pratiques présentent plusieurs limites, notamment la duplication des données, le manque de traçabilité des opérations, l'absence de contrôle d'accès rigoureux et une forte exposition aux erreurs humaines.

Les systèmes existants souffrent également d'une administration insuffisante des bases de données. L'absence de politiques claires de gestion des utilisateurs, de sauvegarde, de restauration et de sécurisation des informations financières fragilise les systèmes et compromet la confidentialité et l'intégrité des données. De plus, la difficulté d'extraire des rapports financiers fiables et à jour rend la prise de décision complexe pour les responsables financiers.

Dans ce contexte organisationnel et technologique, il devient indispensable de mettre en place une solution moderne reposant sur un système de gestion de base de données robuste, capable de centraliser les informations financières, de garantir leur sécurité et d'assurer une administration efficace. C'est dans cette optique que s'inscrit le présent projet d'analyse, conception et administration d'une base de données de gestion des finances sous Oracle.

2. Justification

Tout projet informatique doit répondre à un besoin réel et apporter des améliorations mesurables. La justification de ce projet repose sur plusieurs constats majeurs :

Sécurité et fiabilité des données financières : les méthodes traditionnelles exposent les organisations à des risques de perte, de modification frauduleuse ou d'accès non autorisé aux données. L'administration d'une base de données Oracle permet de mettre en place des mécanismes avancés d'authentification, d'autorisation et de contrôle d'accès.

Centralisation et cohérence des informations : l'utilisation d'une base de données unique permet d'éliminer la dispersion des informations financières et d'assurer leur cohérence, facilitant ainsi leur exploitation.

Traçabilité et transparence : chaque opération financière est historisée, ce qui permet de suivre les actions réalisées, d'identifier les responsables et de renforcer la transparence dans la gestion des finances.

Optimisation de la gestion financière : la structuration des données facilite l'analyse financière, la production de rapports fiables et l'aide à la prise de décision stratégique.

Administration efficace du système : Oracle offre des outils performants pour la gestion des utilisateurs, des rôles, des privilèges, des sauvegardes et de la restauration des données, garantissant la disponibilité du système.

Pertinence dans le contexte camerounais : face au manque de solutions financières bien administrées dans de nombreuses structures locales, ce projet apporte une réponse adaptée aux réalités organisationnelles et techniques du pays.

Bénéfices attendus : amélioration de la fiabilité des données financières, réduction des erreurs, sécurisation des informations sensibles, gain de temps dans la production des rapports et amélioration globale de la gouvernance financière.

II. Les objectifs de l'étude / du projet

1. Objectif général

L'objectif général de ce projet est de concevoir, analyser et administrer une base de données de gestion des finances sous Oracle, afin de garantir la sécurité, la fiabilité, la traçabilité et la disponibilité des données financières au sein d'une organisation.

2. Objectifs spécifiques

Les objectifs spécifiques définissent les actions nécessaires pour atteindre l'objectif général :

- Mettre en place une base de données financière structurée
- Concevoir un modèle de données adapté à la gestion des finances.
- Centraliser les informations relatives aux recettes, dépenses, budgets et transactions.
- Assurer la sécurité et la fiabilité des données
- Mettre en œuvre des mécanismes d'authentification et d'autorisation Oracle.
- Gérer les rôles et privilèges des utilisateurs.
- Protéger les données contre les accès non autorisés.
- Garantir la traçabilité des opérations financières
- Historier toutes les transactions.
- Mettre en place des mécanismes de journalisation et d'audit.
- Faciliter l'analyse et la prise de décision
- Générer des rapports financiers fiables.
- Permettre un suivi clair des budgets et des flux financiers.
- Optimiser l'administration de la base de données
- Mettre en place des sauvegardes automatiques.
- Assurer la restauration des données en cas d'incident.
- Optimiser les performances du système.

III. Expression des besoins de l'utilisateur

1. Besoins fonctionnels

Les besoins fonctionnels définissent les actions que le système doit être capable d'exécuter.

Tableau : Besoins fonctionnels

Acteurs	Titre	Objectifs
Utilisateur (Comptable / Responsable financier)	Gérer les comptes	Permettre l'inscription, l'authentification et la gestion des profils utilisateurs
Comptable	Enregistrer les recettes	Saisir et consulter les recettes financières
Comptable	Enregistrer les dépenses	Saisir et consulter les dépenses
Responsable financier	Gérer les budgets	Créer, modifier et suivre les budgets
Utilisateurs autorisés	Consulter les transactions	Visualiser l'historique des opérations financières
Administrateur	Gérer les utilisateurs	Créer, modifier et supprimer les comptes et rôles
Administrateur	Administrer la base de données	Gérer les sauvegardes, restaurations et performances

Direction Produire des rapports financiers et statistiques	Générer des rapports	Produire des rapports financiers et statistiques
---	----------------------	---

Tableau 2;besoin fonctionnel

2. Besoins non fonctionnels

Les besoins non fonctionnels définissent les critères de qualité du système.

Sécurité informatique

- ❖ Authentification des utilisateurs
- ❖ Confidentialité des données financières
- ❖ Intégrité des informations
- ❖ Disponibilité permanente des données
- ❖ Non-répudiation des actions effectuées

Maintenabilité

- ❖ Le système doit permettre des mises à jour faciles et l'ajout de nouvelles fonctionnalités sans perturber son fonctionnement.

Robustesse

- ❖ La base de données doit fonctionner correctement même en cas de forte charge ou de panne partielle.

Réutilisabilité

- ❖ Les structures de données et certains modules doivent pouvoir être réutilisés dans d'autres systèmes de gestion.

Ergonomie

- ❖ Les interfaces de gestion et d'administration doivent être claires, simples et adaptées aux utilisateurs, même non spécialistes des bases de données.

Dossier 3 : analyse

Résumé :

Le dossier d'analyse a pour but d'affiner le choix de la méthode à utiliser tout au long du projet. Il permet dans la même lancée de faire comme son nom l'indique une analyse détaillée du projet à mettre en œuvre.

Aperçu :

- VI. PRESENTATION DU THEME
- VII. ETUDE L'EXISTANT
- VIII. CRITIQUE DE L'EXISTANT
- IX. PROBLEMATIQUE
- X. PROPOSITION DE LA SOLUTION

I. Méthode et langage d'analyse

L'analyse est une étape fondamentale dans la conception d'un logiciel. C'est la base de tous travaux de réalisation de système d'information. Un système d'information est un système organisé de ressources, de personnes, et de structures qui évoluent dans une organisation et dont le comportement coordonné vise à atteindre un but commun. Plusieurs méthodes de langages ont été développées pour faciliter et normaliser l'analyse et la conception des systèmes d'information parmi lesquels nous avons principalement UML et MERISE.

1. Etude comparative UML et MERISE

MERISE (Méthode d'Étude et de Réalisation Informatique pour les Systèmes d'Entreprise) est une méthode d'analyse, de conception structurelle et de réalisation des systèmes d'informations très utiles notamment dans les entreprises françaises. Elle est basée sur la séparation des données et des traitements à effectuer en plusieurs modèles conceptuels et physiques. Son but principal est d'arriver à concevoir un système d'information (SI). C'est une méthode systémique d'analyse et de conception de SI qui propose de considérer le système réel selon deux points de vue : une vue statique (données) et une vue dynamique (traitement).

UML (Unifie Modelling Language) quant à lui, est un langage de modélisation des systèmes standards, qui utilise des diagrammes pour présenter chaque aspect d'un système en s'appuyant sur la notion d'orienté objet qui est un véritable atout pour ce langage. UML propose donc une approche différente de celle de MERISE en ce sens qu'il associe les données aux traitements. En effet, avec UML, centraliser les données d'un type de traitement associé permet de limiter les points de maintenance dans le code et faciliter l'accès à l'information en cas d'évolution de logiciel. De plus, UML décrit la dynamique du système d'information comme un ensemble d'opérations attachées aux objets du système.

Table 2: Tableau comparatifs UML et MERISE

MERISE	UML
Méthode d'étude et de réalisation Informatique pour les systèmes d'entreprises.	Unified Modeling Language
Est une méthode systémique d'analyse et de conception de systèmes d'information. C'est-à-dire qu'elle utilise une approche systémique.	N'est cependant pas une méthode, mais Plutôt un langage de modélisation objet qu'il faut associer une démarche pour en faire une méthode. C'est le cas de la méthode 2TUP ; RUP et XP.
Propose de considérer le système réel selon deux points de vue : Une vue statique (données). Une vue dynamique (traitements). C'est-à-dire qu'avec la méthode Merise, nous avons une étude séparée des données et des traitements.	Propose une approche différente de celle de Merise en ce sens qu'il associe les données et les traitements. Car avec UML, centraliser les données d'un type et les traitements associés permet de limiter les points de maintenance dans le code et facilite l'accès à l'information en cas d'évolution du Logiciel. De plus ; UML décrit la dynamique du système d'information comme un ensemble d'opérations attachées aux objets du système.

Tableau 3;etude comparative uml et merise

2. Choix et présentation de la démarche d'analyse

Après cette étude comparative entre les méthodes UML et MERISE, notre choix sera porté sur la méthode UML.

UML se définit comme étant un langage de modélisation graphique et textuel destiné à l'architecture, la conception et la mise en œuvre de systèmes logiciels complexes par leurs structures aussi bien que leurs comportements, concevoir des solutions et communiquer des points de vue. L'UML utilise les points forts de ces trois approches pour présenter une méthodologie plus cohérente et plus facile à utiliser. Il représente les meilleures pratiques de création et de documentation des différents aspects de la modélisation des systèmes logiciels et d'entreprise. UML unifie à la fois les notions et les concepts orientés objets. Il ne s'agit pas d'une simple notation graphique, car les concepts transmis par un diagramme ont une sémantique précise et sont porteurs de sens au même titre que les mots d'un langage. C'est un langage standard de modélisation des systèmes d'informations à objet.

La version d'UML utilisée ici est la version 2.5.1 qui compte quatorze (14) diagrammes, et une division du système en deux (02) grandes vues :

- ❖ La vue statique composée des diagrammes UML structurels qui représente le système physiquement et comporte six (07) diagrammes
 - Diagramme de classe
 - Diagramme de composants
 - Diagramme de structures composite
 - Diagramme de déploiement
 - Diagramme d'objets
 - Diagramme de packages
 - Diagramme de profil
- ❖ La vue dynamique constituée des diagrammes UML comportementaux qui représente les interactions effectuées dans le système, elle comporte sept (07) diagrammes :
 - Diagramme d'activité

- Diagramme de communication
- Diagramme global d'interaction
- Diagramme de séquence
- Diagramme d'états-transition
- Diagramme de temps
- Diagramme de cas d'utilisation

UML étant un langage formel et normalisé, il est un bon support de communication dans l'élaboration d'une solution informatique, sa polyvalence et sa souplesse font de lui un langage universel. Pour développer des solutions, on a besoin de lui associer une méthode générique qui s'attache à ses diagrammes, par exemple la méthode Agile. Il en existe plusieurs: Scrum, XP (Extreme Programming), RUP (Rational Unified Process), FDD (Feature Driven Development), etc...

Dossier 4 : Conception

Résumé :

Le dossier de conception permet de modéliser dans son ensemble la solution proposée et de recueillir les informations nécessaires à la mise sur pied d'une base de données complexes et efficaces. De ce fait il prévoit le système futur.

Aperçu :

XI. DIAGRAMME DE CLASSE

I. Diagramme de classe

Le diagramme de classe met en évidence d'éventuelles relations entre ces classes. Le diagramme de classes comporte 6 concepts : **classe**, **attribut**, **identifiant**, **relation**, **opération**, **généralisation/spécialisation**. Ce diagramme est constitué des éléments suivants :

- **Classe** : il s'agit de la description abstraite à un ensemble d'objet : elle définit leurs structures, leurs comportements et leurs relations.
- **L'attribut** : Il représente la modélisation d'une information élémentaire représentée par son nom et format.

UML définit 3(trois) niveaux de visibilité pour les attributs :

- **Public (+)** : l'élément est visible par tous les clients de la classe.
- **Protégé (#)** : l'élément est visible que par les sous-classes ou les classes filles.
- **Privé (-)** : l'élément n'est visible que par les objets de la classe auxquels il appartient.
- **L'identifiant** : c'est un attribut particulier, qui permet de repérer de façon unique chaque objet, instance de la classe.
- **Multiplicité** : elle définit le nombre d'instance de la classe, elle est définie par un nombre entier ou un intervalle de valeur (1...1, 0...1, 1...*, 0...*).
- **Associations** : Une association est une relation entre deux classes (association binaire) ou plus (association n-aire), qui décrit les connexions structurelles entre leurs instances.

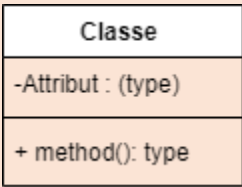
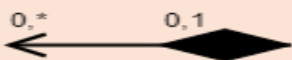
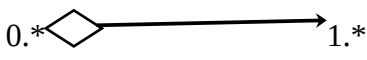
Il existe plusieurs types de relations dont les plus connues sont les suivants :

- **Généralisation** : Relation d'héritage, dans laquelle les objets de l'élément spécialisé (Classe enfant) peuvent remplacer les objets de l'élément général (Classe parent)

- **Agrégation** : utilisé pour les relations de types « contenant / contenue »
- **La composition** : La composition est un cas particulier de l'agrégation dans laquelle la vie des composants est liée à celle des agrégats.

Le tableau suivant nous donne une liste assez détaillée du formalisme du diagramme de classe en un tableau :

Table 3: Composants du diagramme de classe

Elément	Représentation
Classe	
Association	
Composition	
Agrégation	
Généralisation	
Multiplicité	0..*, 0...1, 1...1, 1...*
Modificateur d'accès	+, -, #, ~, /

Formalisme du diagramme de classe :

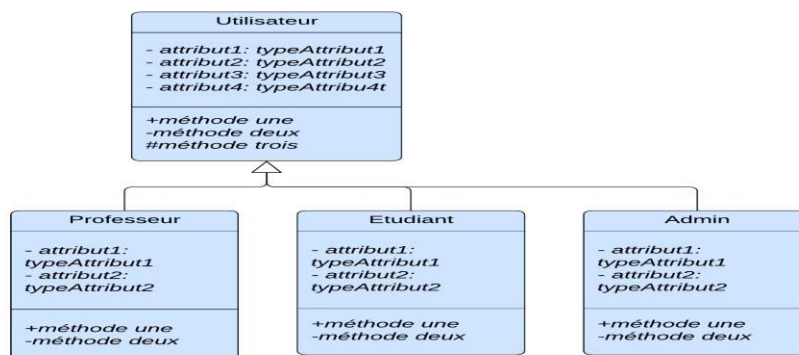


Figure 1:Formalisme du diagramme de classe

Le diagramme de classe de notre projet est le suivant :

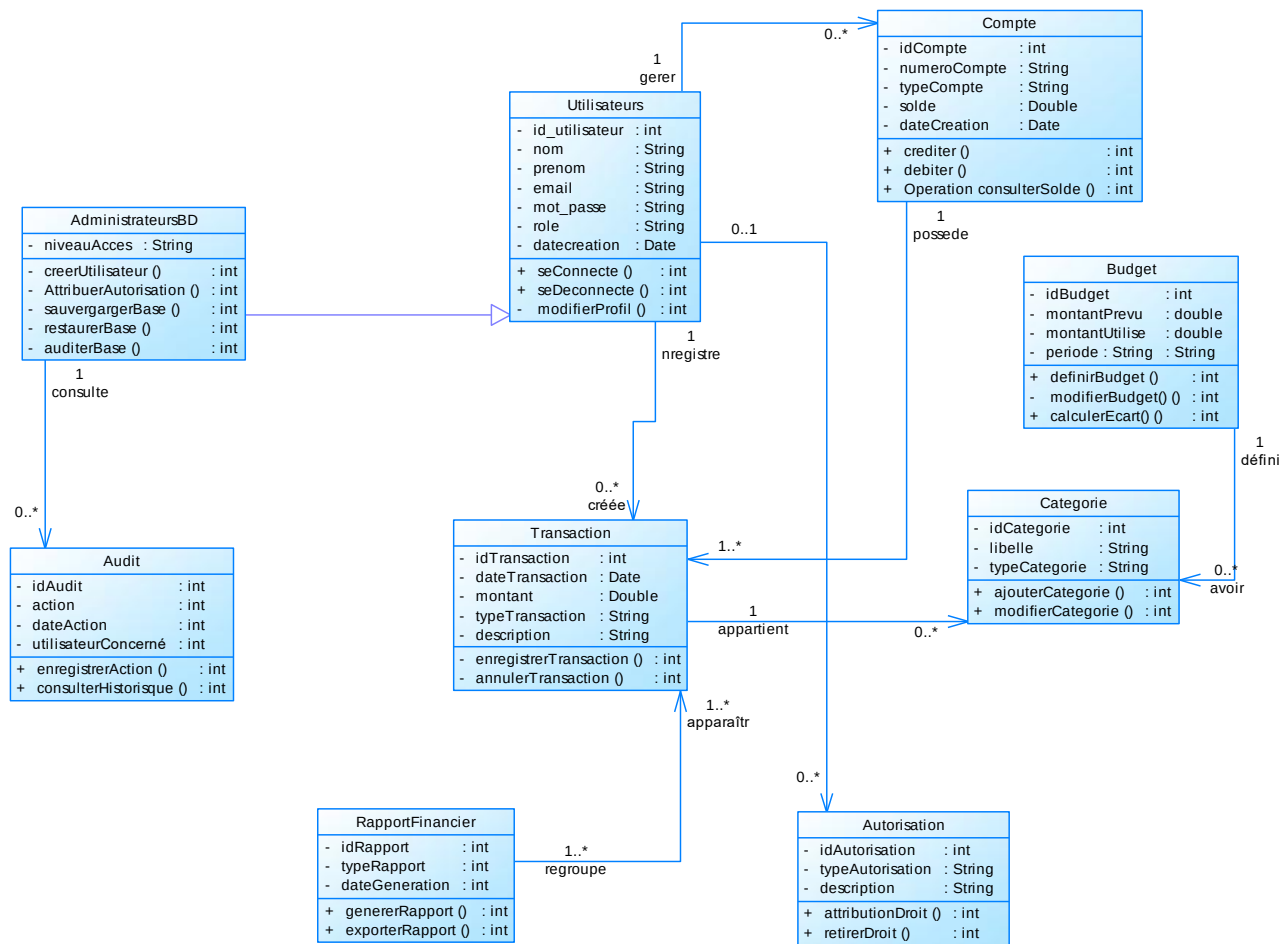


Figure 2:diagramme de classe du système

Dossier 5 : Dossier de Réalisation :

Résumé :

La réalisation consiste à la mise en œuvre de projets dans un langage de programmation conformément aux spécifications définies dans le dossier précédent. Elle renferme en son sein les diagrammes de composant et de déploiement. A la sortie de cette partie, il sera produit une documentation de programmation expliquant l'architecture de la base de données et l'architecture de notre code.

Aperçu :

XII. PRESENTATION DE L'ENVIRONNEMENT LOGICIEL ET TECHNOLOGIQUE

I. Présentation de l'environnement logiciel

1. Environnement logiciel

Les logiciels utilisés dans le cadre de la réalisation de projet sont résumés dans le tableau ci-dessous :

Table 21: Logiciels utilisés

Nom du logiciel	Version	Utilisation	Logo
Oracle 11G	11.2.0.4	Système de Gestion de Base de Données Relationnelle (SGBDR) conçu pour stocker, gérer et sécuriser de grands volumes de données d'entreprise	
SQL développer	24.3.1	environnement de développement gratuit qui facilite la conception, la gestion et l'optimisation de bases de données Oracle	
Power AMC	15.1	Atelier génie logiciel utilisé pour la modélisation de la solution.	

Tableau 4;different logiciel

Dossier 6 : Test de fonctionnalités

Résumé :

Un test désigne une procédure de vérification partielle d'un système. Son objectif principal est d'identifier un nombre maximal de comportements problématiques du logiciel. Il permet ainsi, dès lors que les problèmes identifiés seront corrigés, d'en augmenter la qualité.

Aperçu :

XIII. FONCTIONNALITES ET TESTS

I. Fonctionnalités

La plateforme de gestion des finances proposée met à disposition plusieurs fonctionnalités majeures permettant une administration efficace, sécurisée et fiable des données financières. Ces fonctionnalités sont organisées en modules principaux afin d'assurer une utilisation fluide, cohérente et conforme aux exigences des organisations.

➤ Inscription et authentification des utilisateurs

Chaque utilisateur du système (administrateur, comptable, responsable financier) peut disposer d'un compte personnel lui permettant d'accéder à la base de données selon son rôle. L'authentification repose sur un mécanisme sécurisé basé sur la gestion des utilisateurs Oracle, avec attribution de rôles et de privilèges spécifiques garantissant un accès contrôlé aux données financières.

➤ Gestion des profils utilisateurs

Les utilisateurs peuvent consulter et mettre à jour certaines informations liées à leur profil (nom d'utilisateur, rôle, paramètres d'accès).

L'administrateur de la base de données assure la gestion des comptes utilisateurs, l'activation ou la désactivation des accès, ainsi que la modification des privilèges en fonction des responsabilités attribuées.

➤ Gestion des comptes financiers

Le système permet la création et la gestion des comptes financiers de l'organisation.

Chaque compte est identifié de manière unique et peut être associé à différentes opérations financières, assurant ainsi une structuration claire et cohérente des données.

➤ Gestion des recettes et des dépenses

Les utilisateurs autorisés peuvent enregistrer les recettes en précisant la source, le montant, la date et la nature de l'opération.

Les dépenses peuvent être saisies avec les informations correspondantes (montant, bénéficiaire, date, catégorie). Ces opérations sont stockées de manière sécurisée dans la base de données Oracle, garantissant leur intégrité et leur traçabilité.

- Suivi et gestion des budgets

La plateforme permet la définition de budgets prévisionnels par période (mensuelle, trimestrielle ou annuelle).

Les dépenses et recettes sont automatiquement associées aux budgets correspondants, facilitant le suivi financier et la comparaison entre prévisions et réalisations.

- Historisation et traçabilité des transactions

Toutes les opérations financières effectuées dans le système sont historiées.

Un mécanisme de journalisation permet de suivre les actions réalisées par chaque utilisateur, assurant ainsi la traçabilité des transactions et renforçant la transparence du système.

- Génération de rapports financiers

Le système permet de produire des rapports financiers tels que :

- États des recettes et dépenses,
- Bilans financiers,
- Rapports budgétaires.

Ces rapports facilitent l'analyse financière et constituent une aide précieuse à la prise de décision.

- Sécurité et contrôle d'accès

La base de données intègre des mécanismes avancés de sécurité, notamment :

- La gestion des rôles et des privilèges Oracle,
- La restriction des accès aux données sensibles,
- La protection contre les accès non autorisés.
- Sauvegarde et restauration des données

Des mécanismes de sauvegarde régulière (backup) et de restauration (recovery) sont mis en place afin de garantir la disponibilité des données financières en cas d'incident, de panne ou de mauvaise manipulation.

➤ Tableau de bord d'administration

L'administrateur dispose d'un tableau de bord lui permettant de :

- Gérer les utilisateurs et leurs rôles,
- Surveiller l'activité de la base de données,
- Contrôler les performances,
- Assurer la maintenance et la sécurité du système.

Dossier 7 : Guide d'installation et guide d'utilisateur

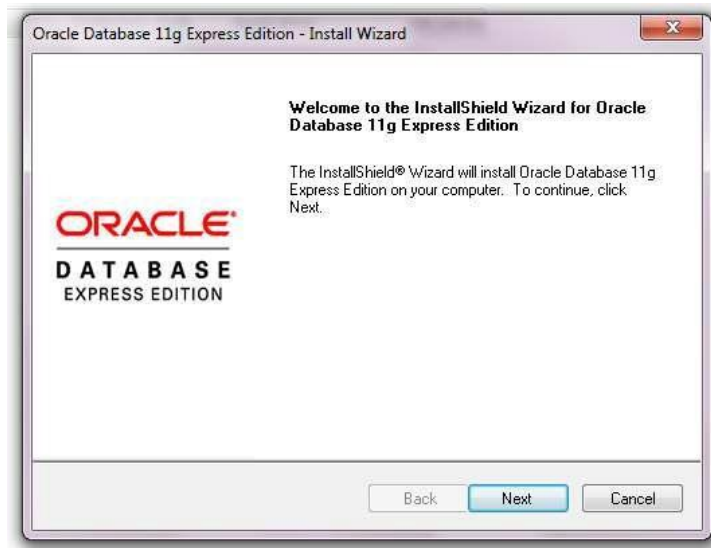
Résumé :

Le manuel d'installation et d'utilisation est le document établi après réalisation d'une application, d'un logiciel ou d'une plateforme. Il renseigne sur la question « comment installer et utiliser le logiciel, l'application ou la plateforme que l'on a devant soi ? »

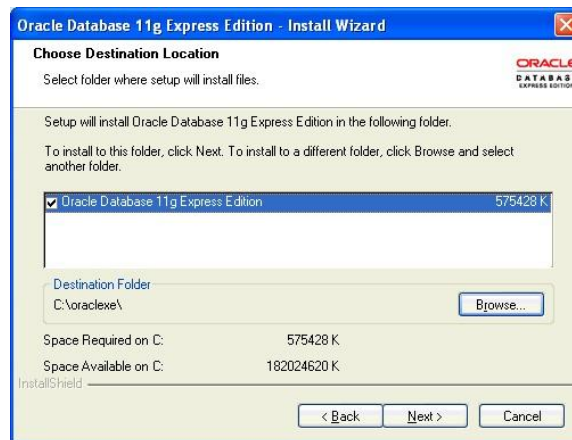
Aperçu :

XIV. GUIDE D'INSTALLATION
XV. CAS PRATIQUE

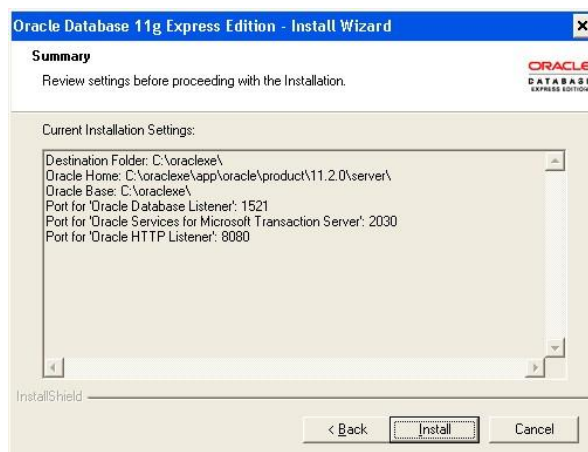
I. installation d'oracle



- Acceptez les termes et les conditions d'utilisation et appuyez sur *Next*.
- Choisissez le répertoire d'installation. Cliquez sur *Browse* pour changer le répertoire par défaut. Attention à ne pas sélectionner un répertoire comprenant des espaces. Cliquez sur *Next*.

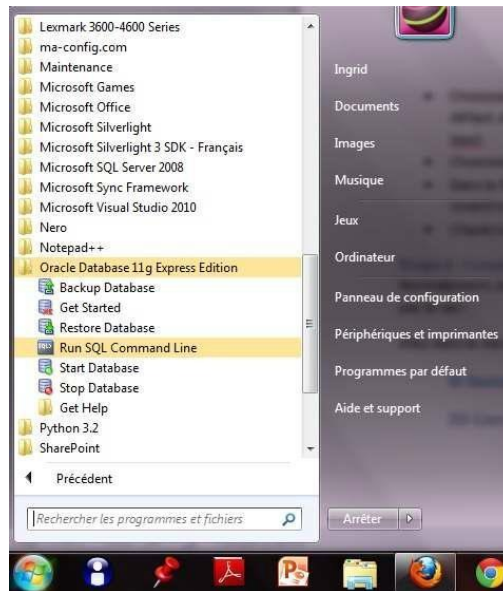


- Choisissez le mot de passe utiliser pour le SYSTEM, et confirmez-le. Cliquez sur *Next*.
- Dans la fenêtre de récapitulation, cliquez sur *Install* si vous êtes satisfait, ou sur *Back* pour revenir en arrière et changer certains éléments.



Normalement, à la fin de l'installation, la base de données est opératrice et est démarré. Si ce n'est pas le cas :

- Allez dans le menu de démarrage - Tous les programmes - Oracle Database 11g Express Edition ;
- Cliquez sur Start Database.



Dans le menu de démarrage - Tous les programmes - Oracle Database 11g Express Edition, cliquez sur *Run SQL Command Line*.

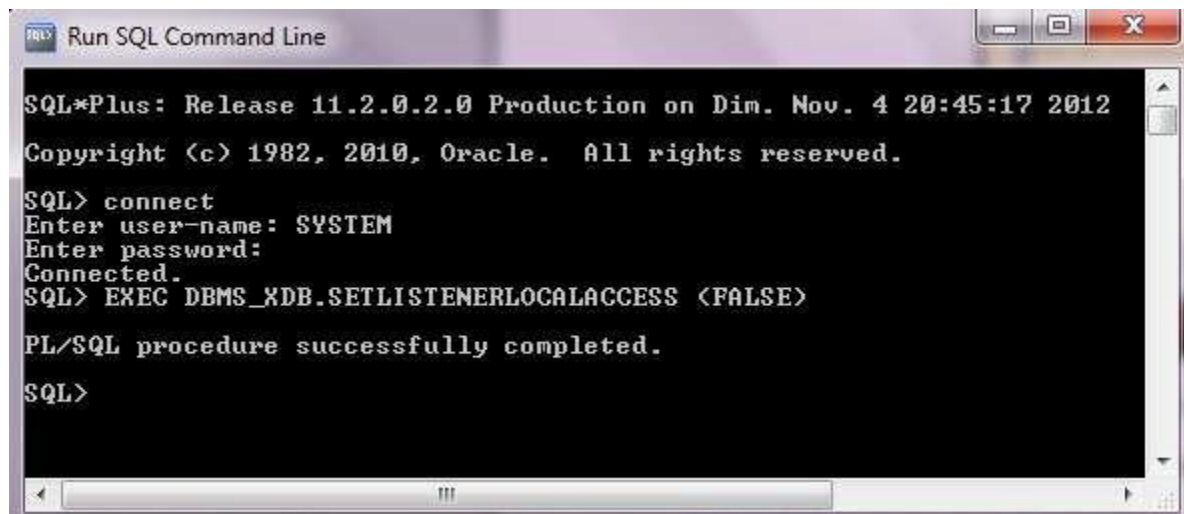
Dans la ligne de commande tapez :

- connect ;
- SYSTEM : pour vous connecter en utilisant le nom d'utilisateur (« user-name ») SYSTEM ; ➤ Le mot de passe que vous avez donné lors de l'installation.

Le message *Connected* apparaît.

Pour que votre base de données soit accessible par des clients à distance, tapez la commande :

```
EXEC DBMS_XDB.SETLISTENERLOCALACCESS (FALSE);
```



```
Run SQL Command Line
SQL*Plus: Release 11.2.0.2.0 Production on Dim. Nov. 4 20:45:17 2012
Copyright (c) 1982, 2010, Oracle. All rights reserved.

SQL> connect
Enter user-name: SYSTEM
Enter password:
Connected.
SQL> EXEC DBMS_XDB.SETLISTENERLOCALACCESS (FALSE)

PL/SQL procedure successfully completed.

SQL>
```

Installation

Les étapes principales sont :

1. Télécharger SQL Developer depuis le site d'Oracle

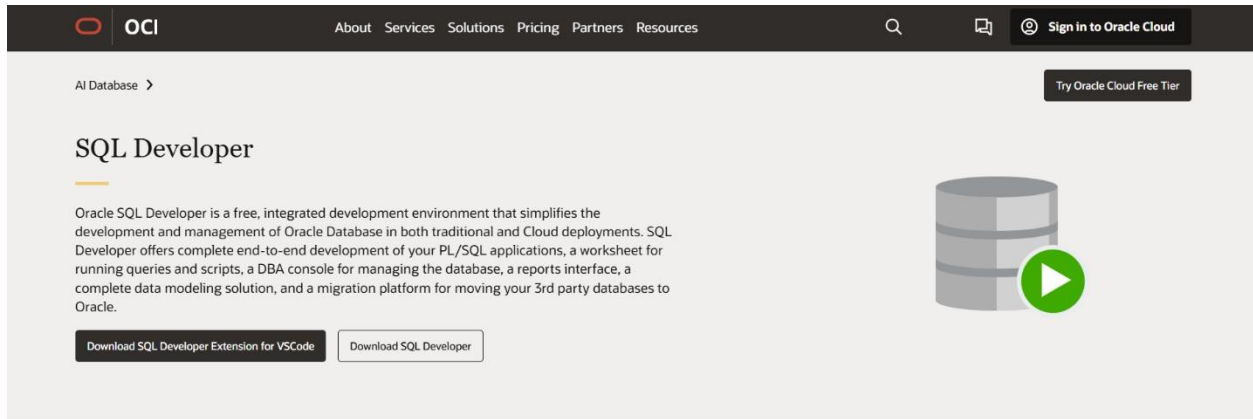


Figure 6; site web SQL developer

2. Lancer SQL Developer

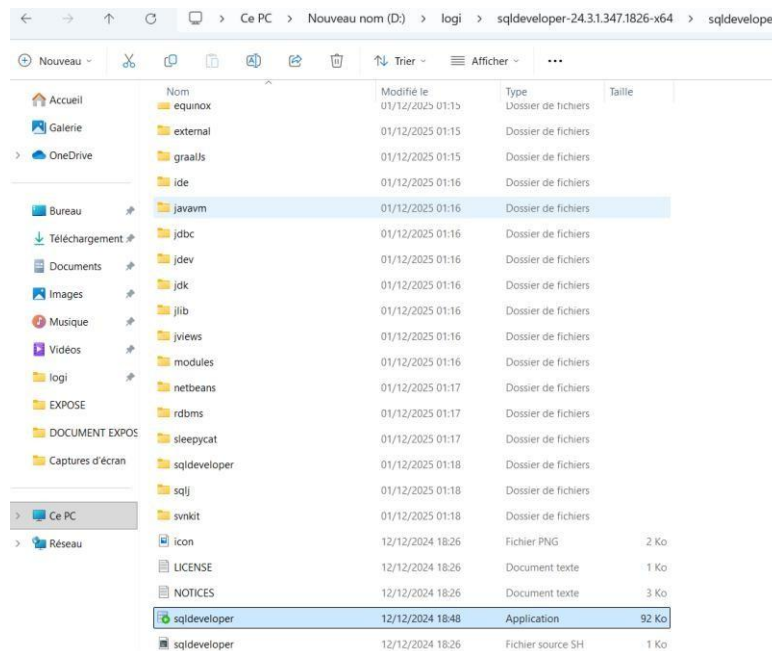


Figure 7:dossier SQL developer

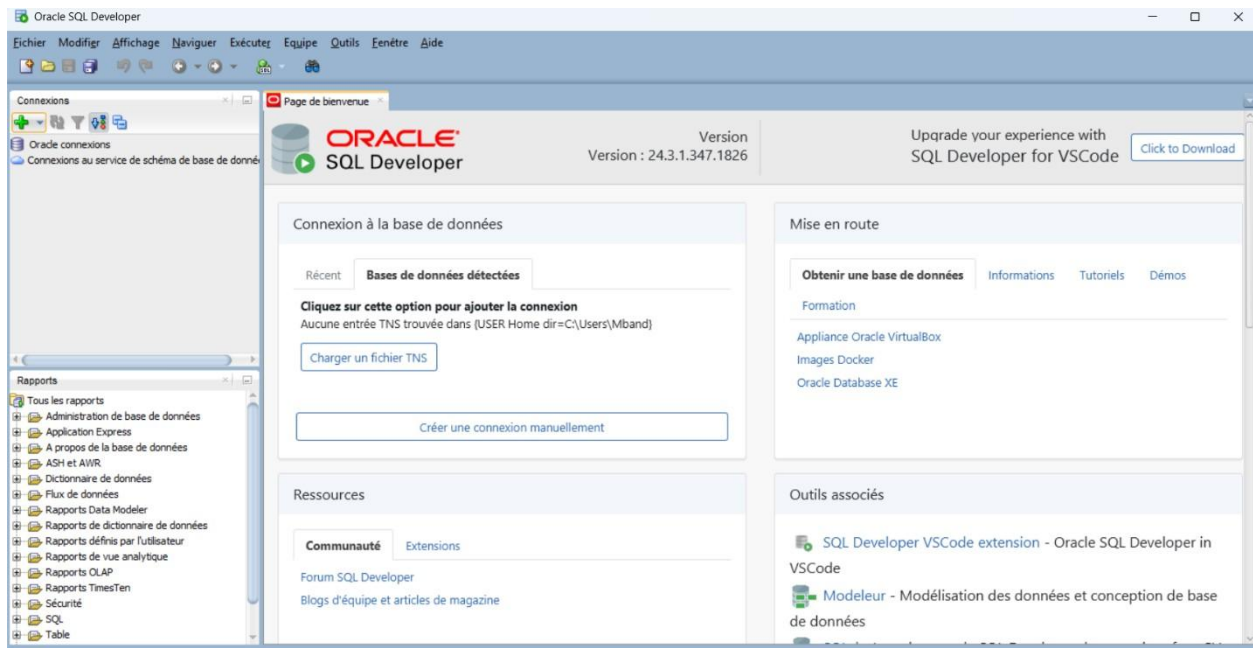


Figure 8: interface oracle

II.4.2 Connexion à une base Oracle

Pour se connecter, il faut :

- Nom d'utilisateur
- Mot de passe
- Nom du service (Service Name)
- Hôte et port

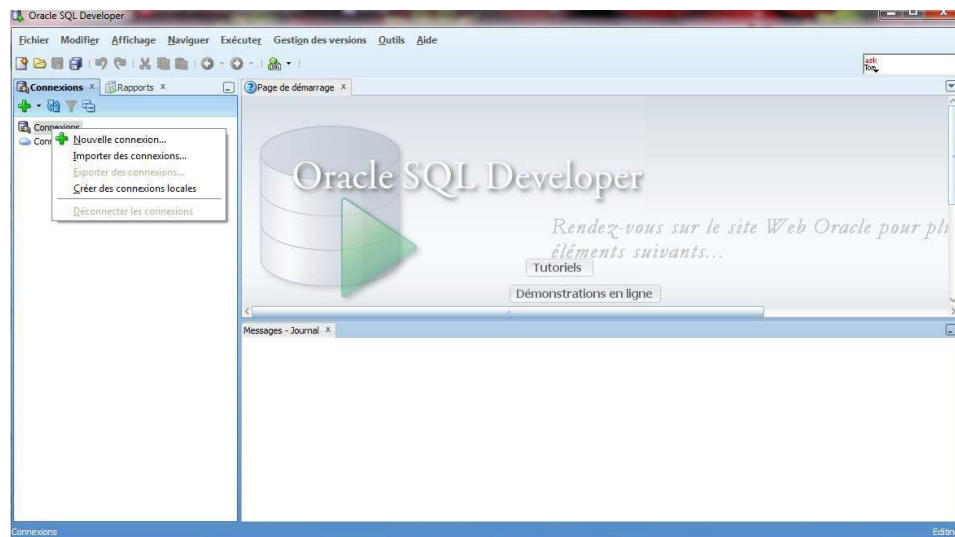


Figure 9: nouvelle connexion

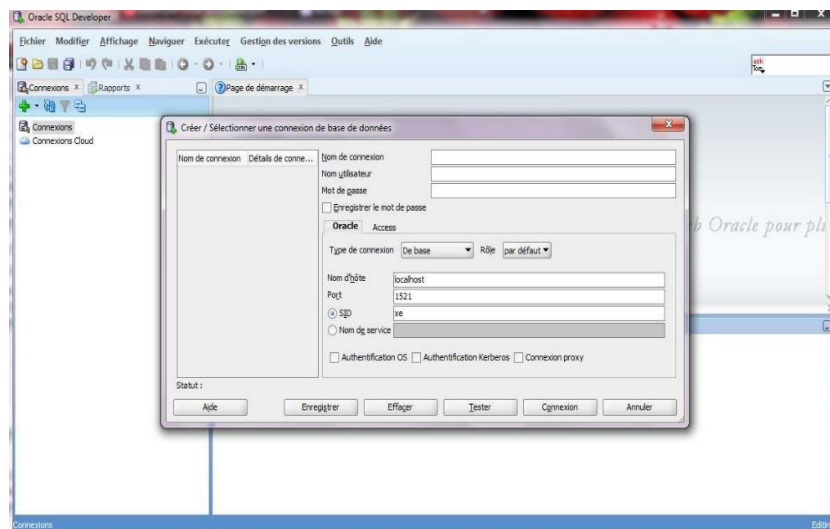


Figure 10: configuration

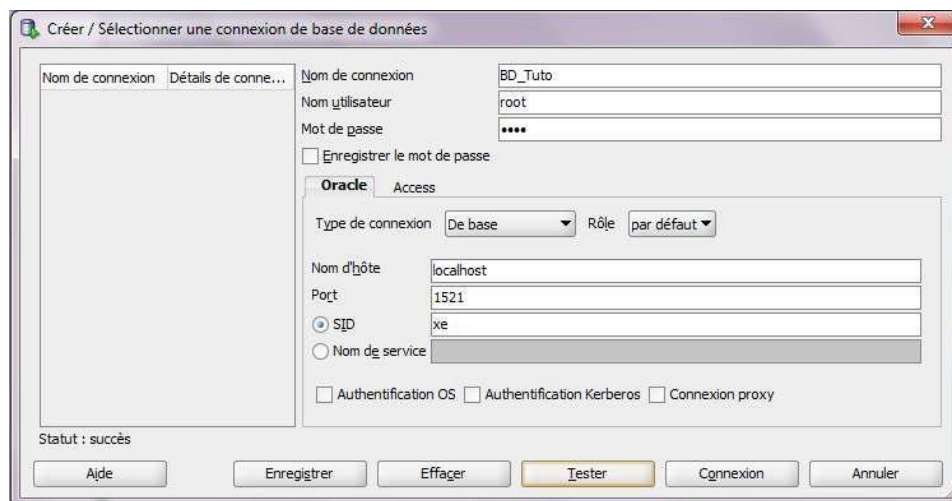


Figure 11:Figure 14 : configuration

I. CAS PRATIQUE

Administration d'une base de données de gestion des finances avec Oracle SQL Developer

ÉTAPE 1 : Lancer Oracle SQL Developer

Ouvre Oracle SQL Developer

Attends que l'interface se charge

Tu dois voir :

À gauche : Connections

Au centre : l'espace de requêtes SQL

ÉTAPE 2 : Créer une connexion à la base de données

Clique sur : New Connection

Renseigne les informations :

Connection Name : FINANCE_DB

Username : system (ou ton utilisateur)

Password : ton mot de passe Oracle

Role : Default

Hostname : localhost

Port : 1521

SID : XE (ou ORCL selon ton installation)

Clique sur Test

Si le message est Success, continue

Clique sur Connect

La connexion apparaît en vert

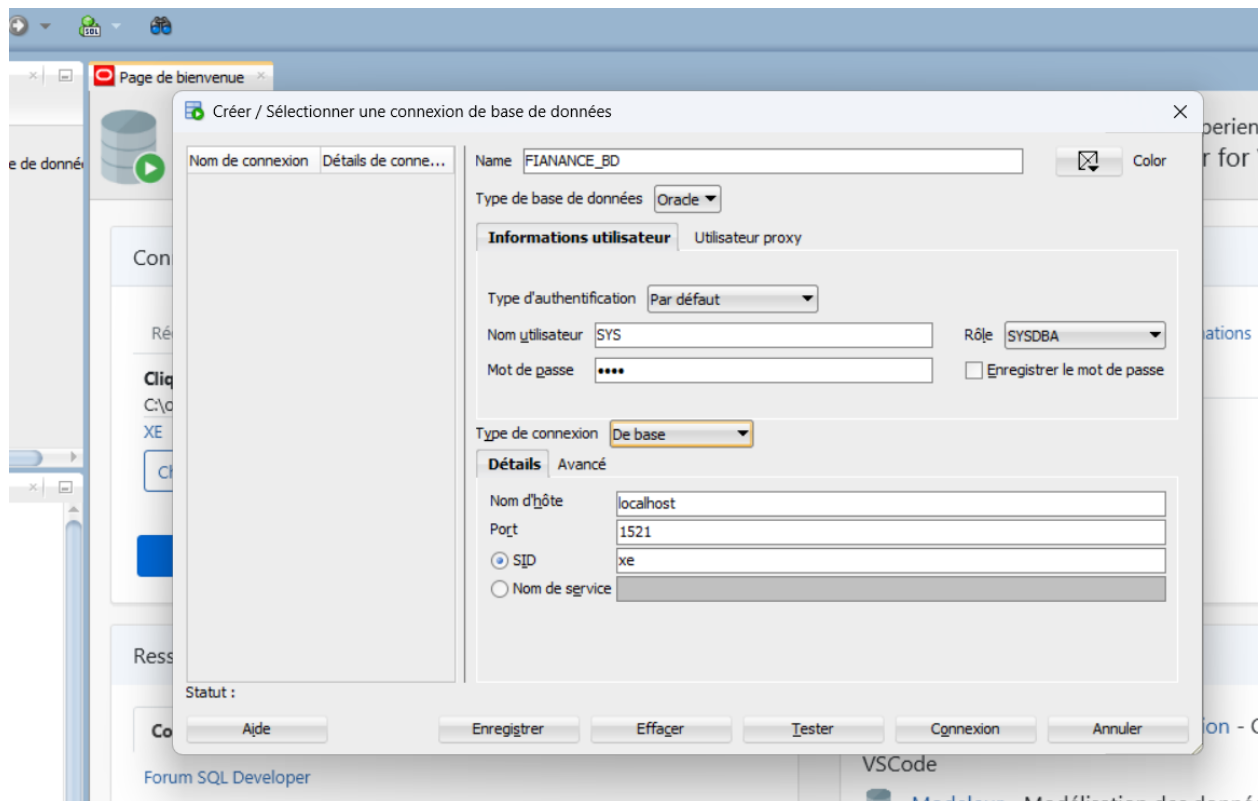


Figure 3: Connexion

ÉTAPE 3 : Créer un utilisateur pour la gestion financière (ADMINISTRATION)

Bonne pratique d'administration : ne pas travailler avec SYSTEM

```
CREATE USER finance_admin IDENTIFIED BY finance123;
```

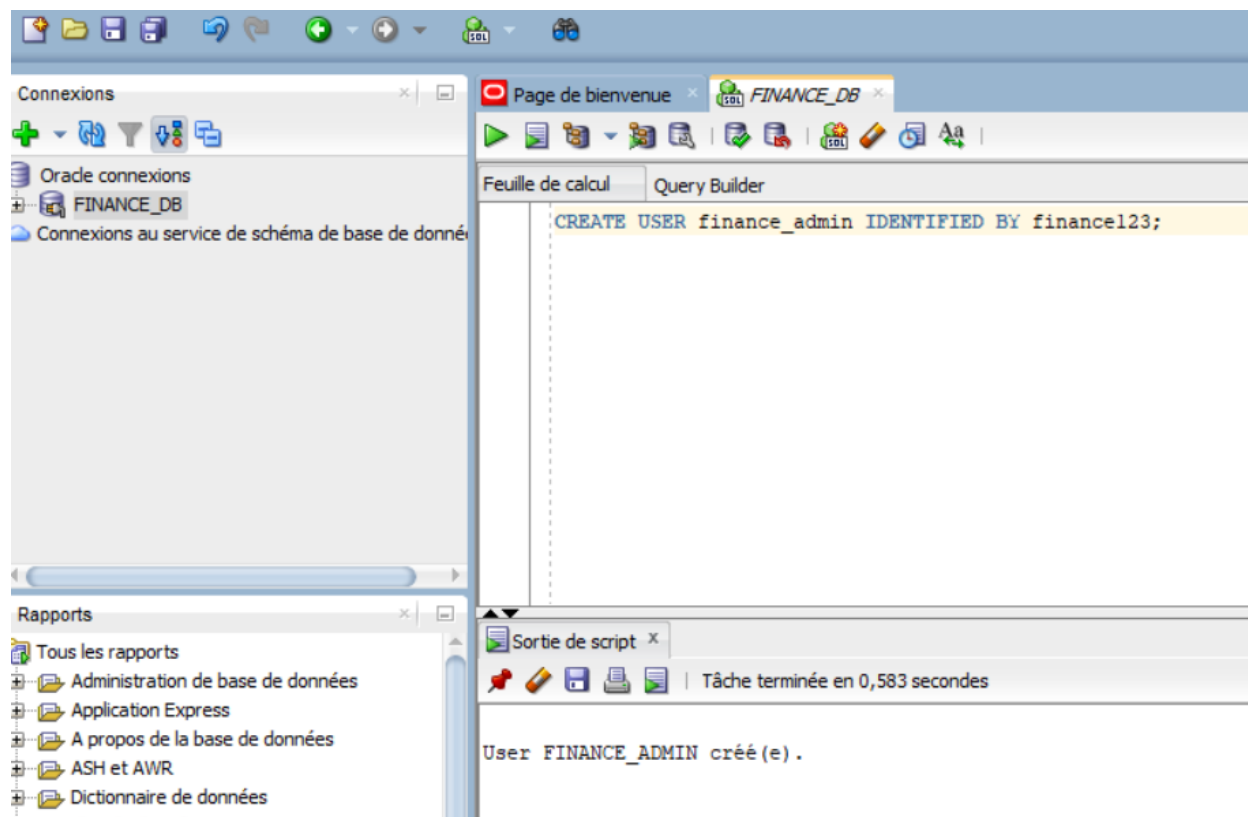


Figure 4:implementation

ÉTAPE 4 : Donner les droits nécessaires (Administration)

GRANT CONNECT, RESOURCE TO finance_admin;

GRANT CREATE TABLE, CREATE VIEW, CREATE SEQUENCE TO finance_admin;

Tu viens d'administrer les droits d'accès (point clé du thème)

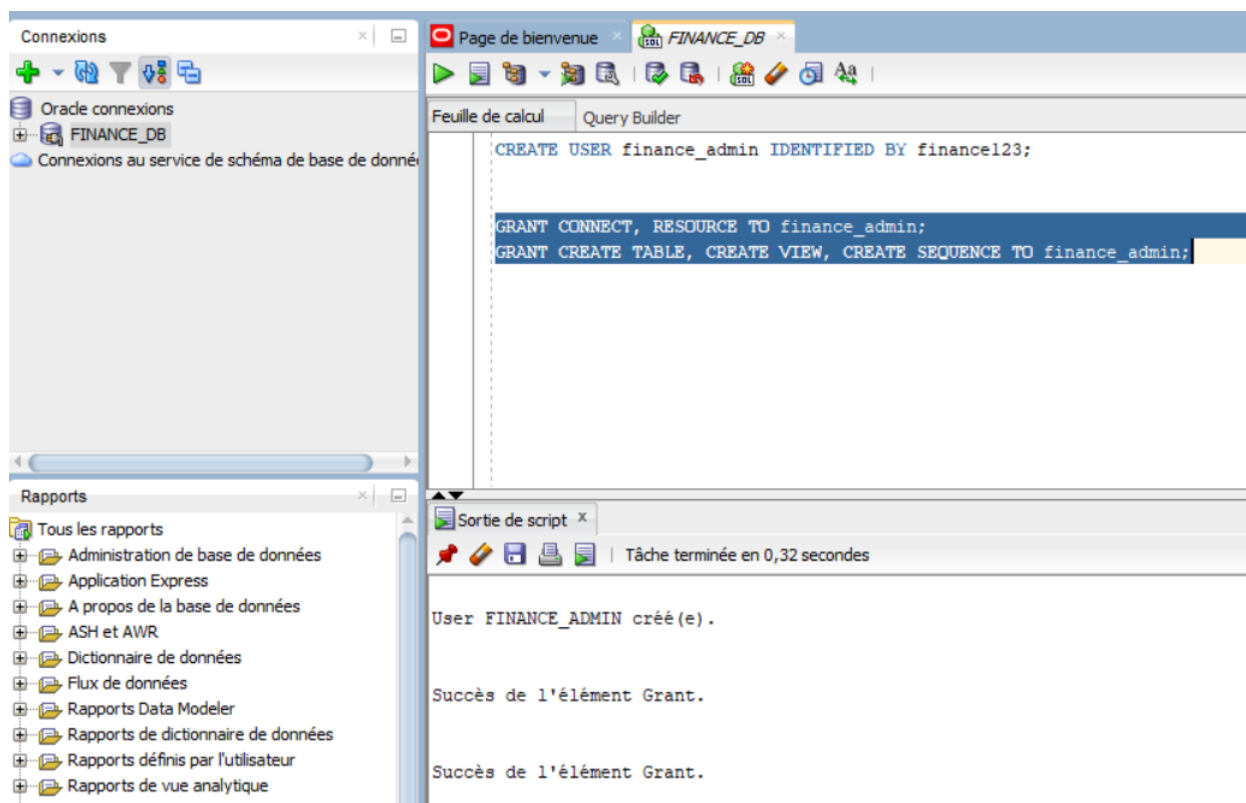


Figure 5:implementation

ÉTAPE 5 : Se connecter avec l'utilisateur financier

Crée une nouvelle connexion

Renseigne :

Username : finance_admin

Password : finance123

Clique sur Connect

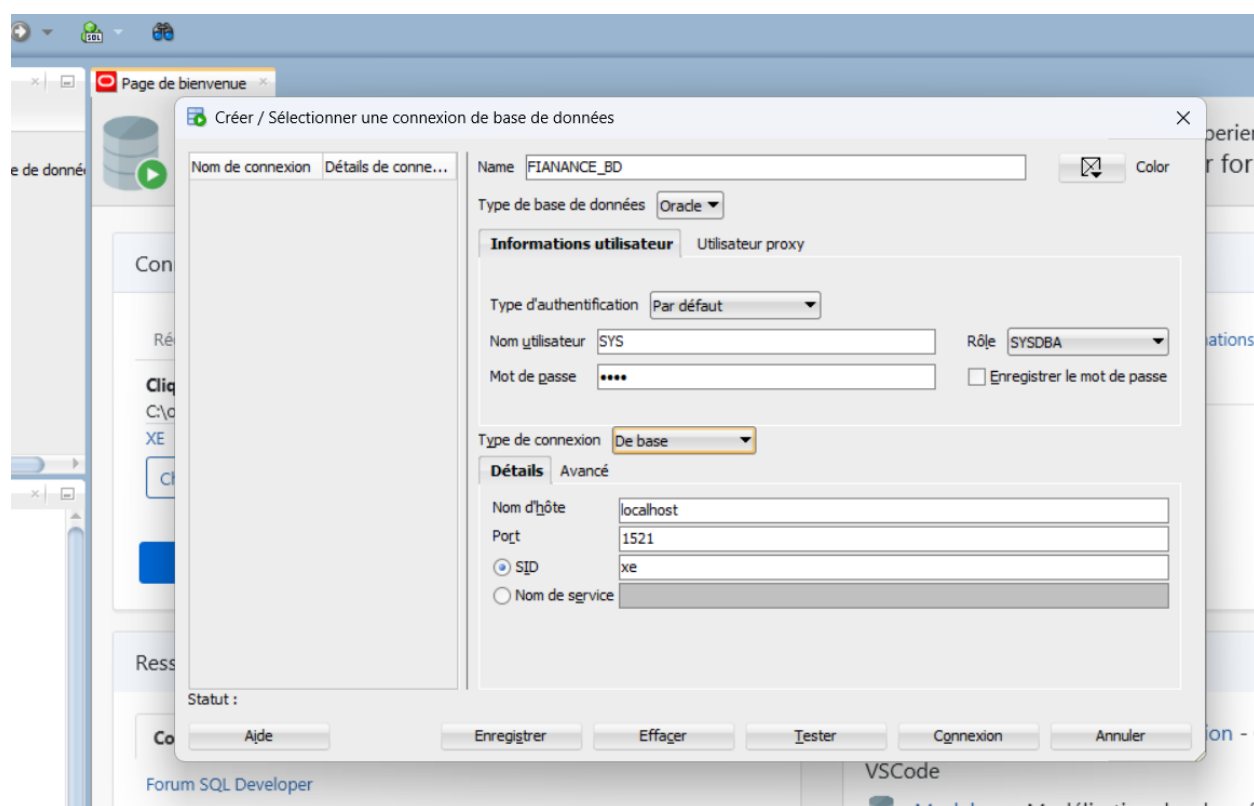


Figure 6:implementation

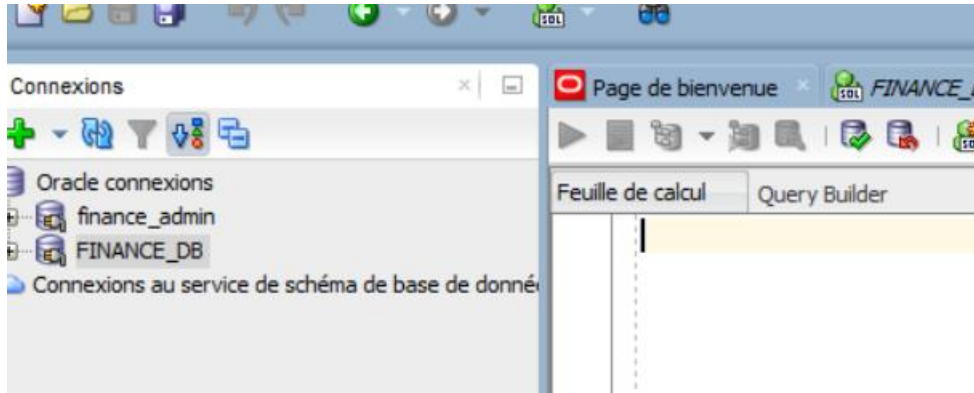


Figure 7:connection admin

ÉTAPE 6 : Création des tables (diagramme de classes → base Oracle)

6.1 Table UTILISATEUR

```
CREATE TABLE utilisateur (
    id_utilisateur NUMBER PRIMARY KEY,
    nom VARCHAR2(50),
    prenom VARCHAR2(50),
    email VARCHAR2(100) UNIQUE,
    mot_de_passe VARCHAR2(100),
    role VARCHAR2(30),
    date_creation DATE
);
```

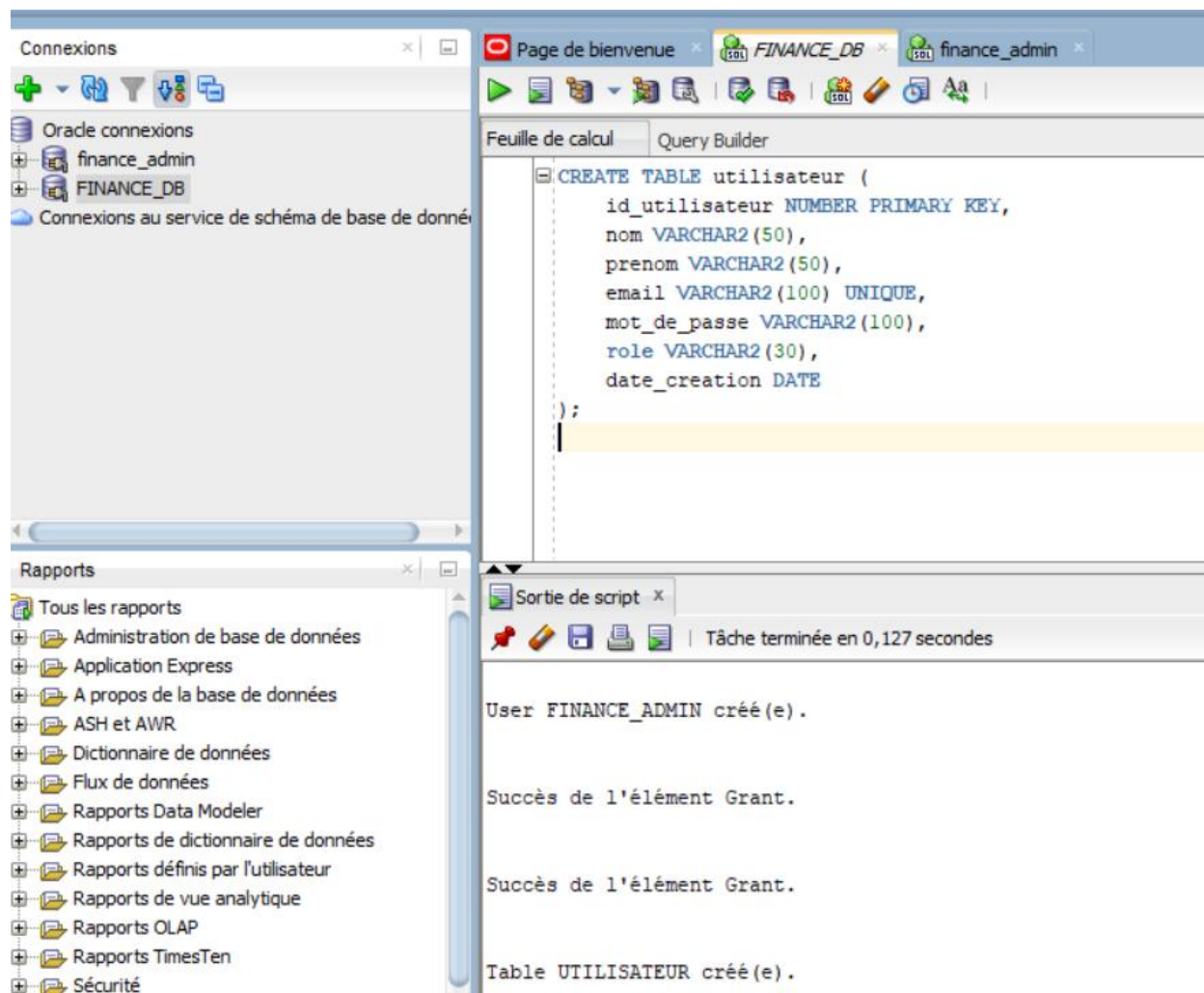


Figure 8:creation de la table user

6.3 Table CATEGORIE

CREATE TABLE categorie (

id_categorie NUMBER PRIMARY KEY,

libelle VARCHAR2(50),

type_categorie VARCHAR2(30)

);

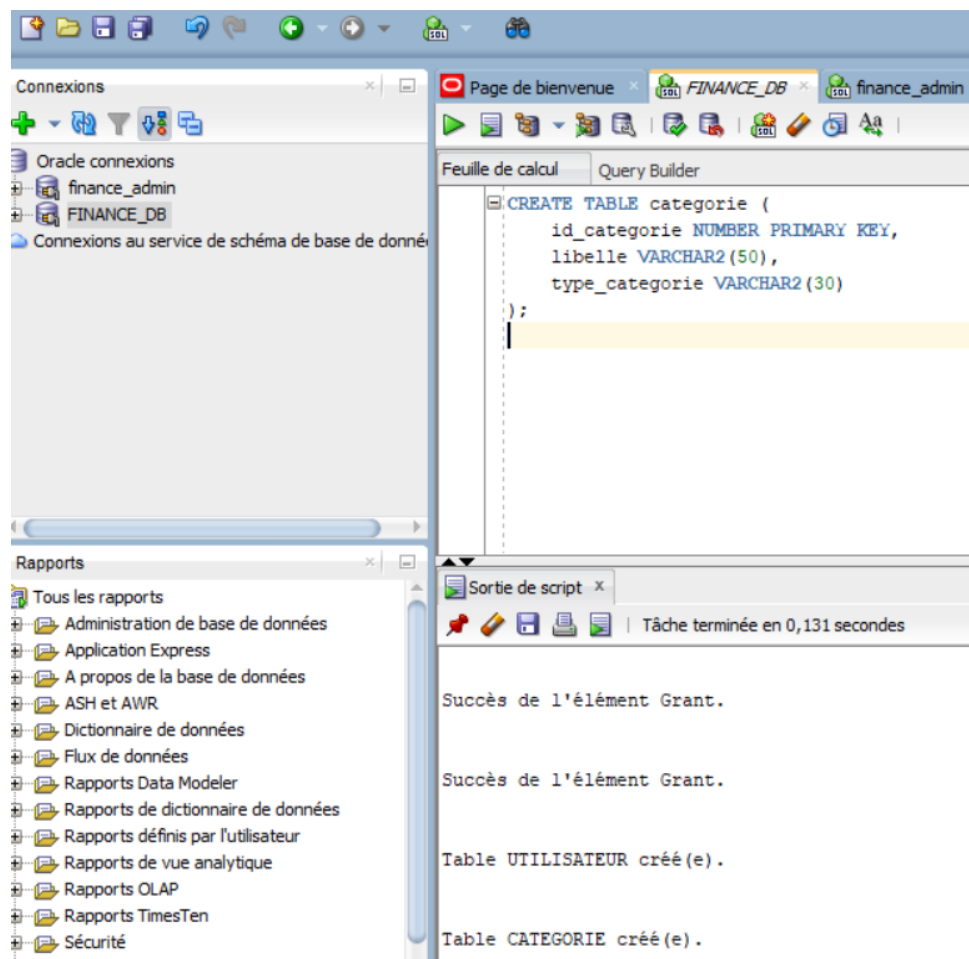


Figure 9:creation de la table categorie

ETAPE DE CREATION DE LA TABLE COMPTE

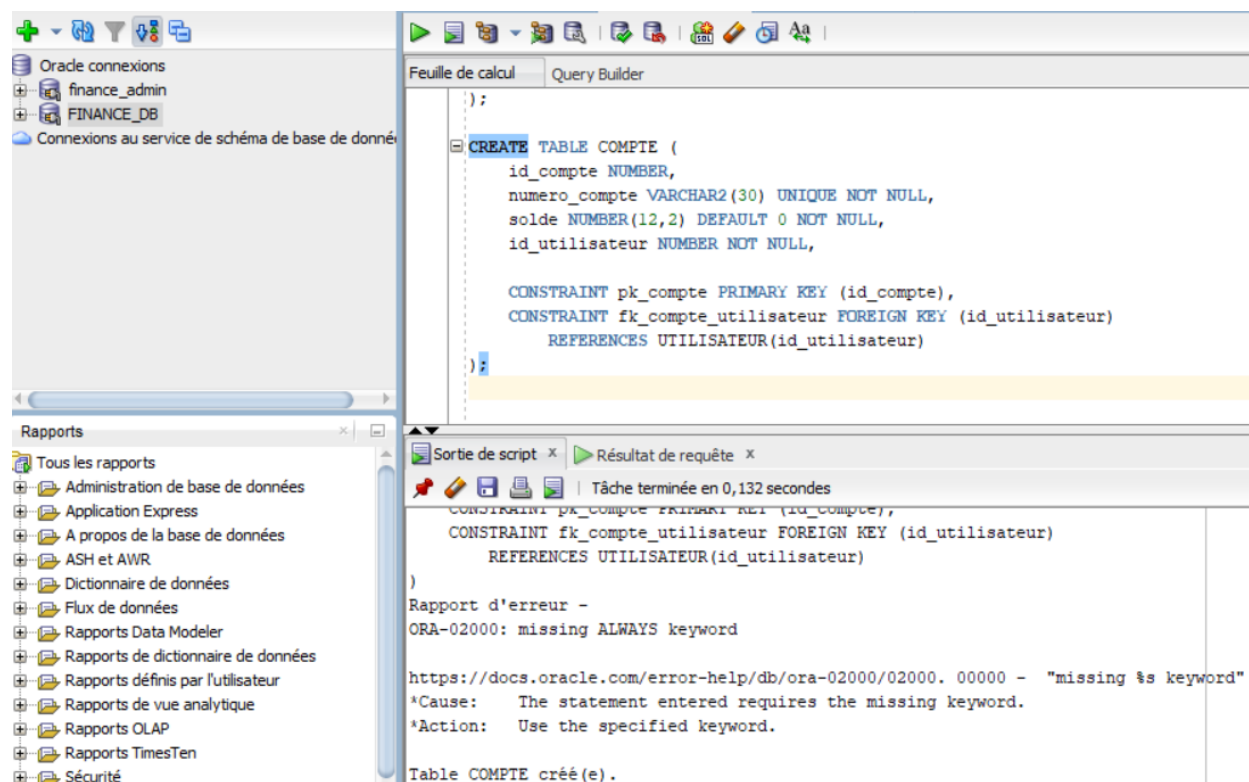


Figure 10: TABLE COMPTE

6.4 Table TRANSACTION

```
CREATE TABLE transaction_financiere (  
    id_transaction NUMBER PRIMARY KEY,  
    date_transaction DATE,  
    montant NUMBER(12,2),  
    type_transaction VARCHAR2(30),  
    description VARCHAR2(100),  
    id_compte NUMBER,  
    id_utilisateur NUMBER,  
    id_categorie NUMBER,  
    CONSTRAINT fk_compte FOREIGN KEY (id_compte) REFERENCES compte(id_compte),  
    CONSTRAINT fk_utilisateur FOREIGN KEY (id_utilisateur) REFERENCES  
utilisateur(id_utilisateur),  
    CONSTRAINT fk_categorie FOREIGN KEY (id_categorie) REFERENCES  
categorie(id_categorie)  
);
```

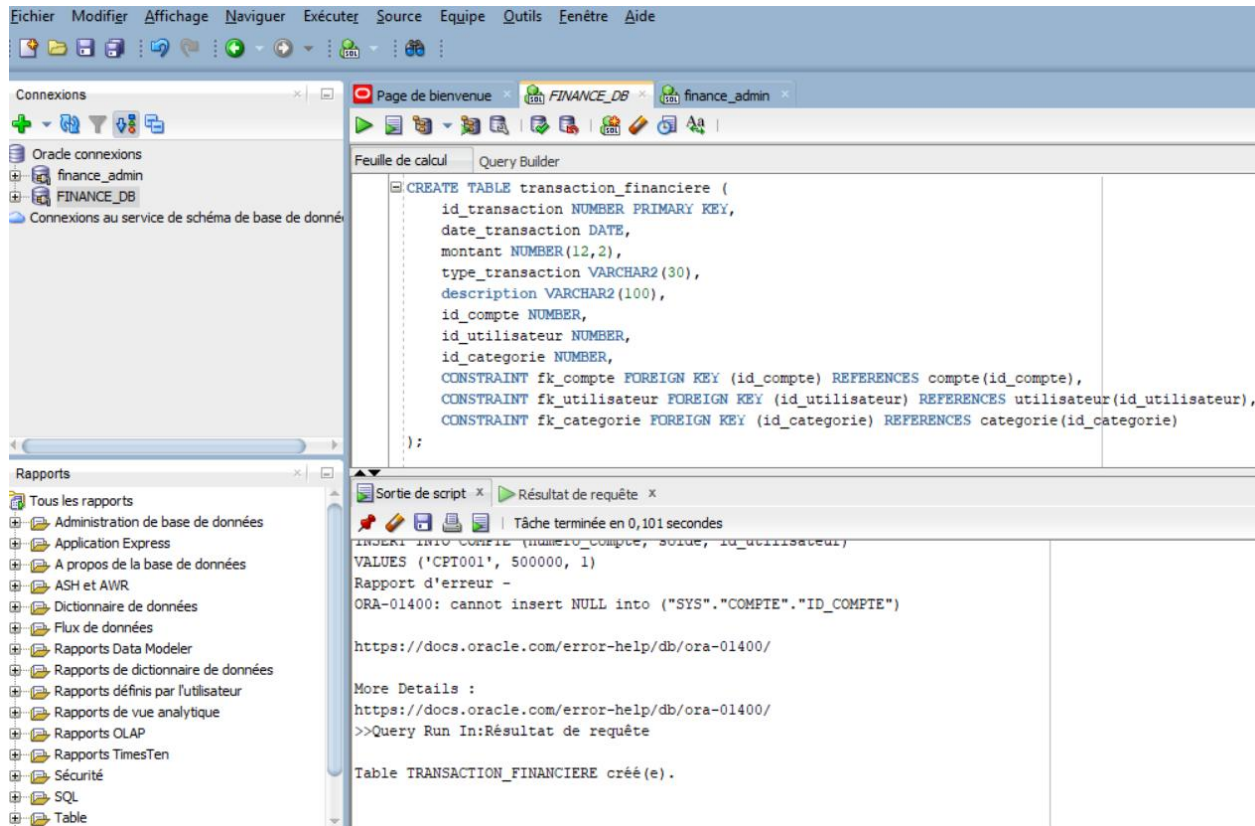


Figure 11:table transaction_finance

6.5 Table AUDIT_log (Administration & sécurité)

CREATE TABLE audit (

id_audit NUMBER PRIMARY KEY,

action VARCHAR2(100),

date_action DATE,

utilisateur VARCHAR2(50)

);

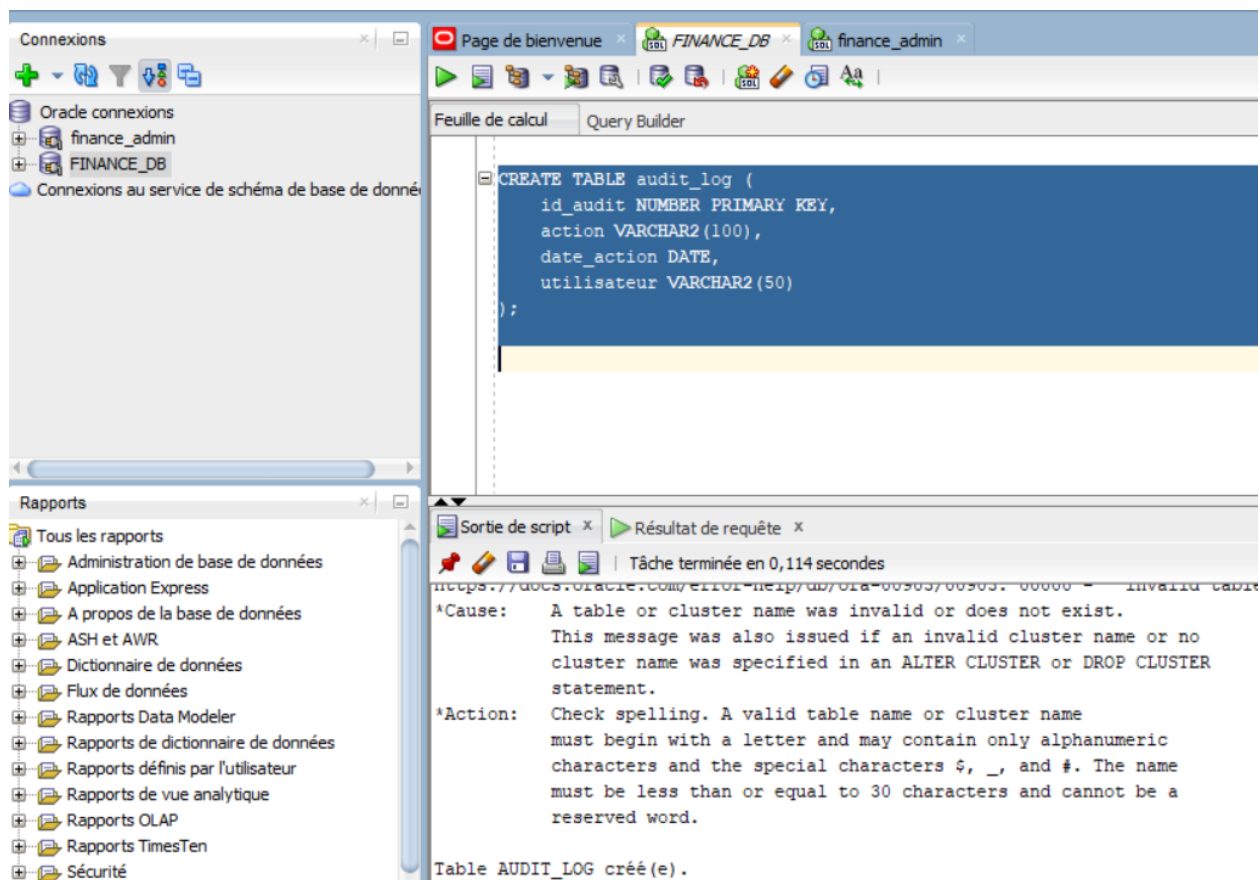


Figure 12:table audit_log

ÉTAPE 7 : Création des SEQUENCES (auto-incrément)

CREATE SEQUENCE seq_utilisateur START WITH 1 INCREMENT BY 1;

CREATE SEQUENCE seq_compte START WITH 1 INCREMENT BY 1;

CREATE SEQUENCE seq_transaction START WITH 1 INCREMENT BY 1;

CREATE SEQUENCE seq_categorie START WITH 1 INCREMENT BY 1;

CREATE SEQUENCE seq_audit START WITH 1 INCREMENT BY 1;

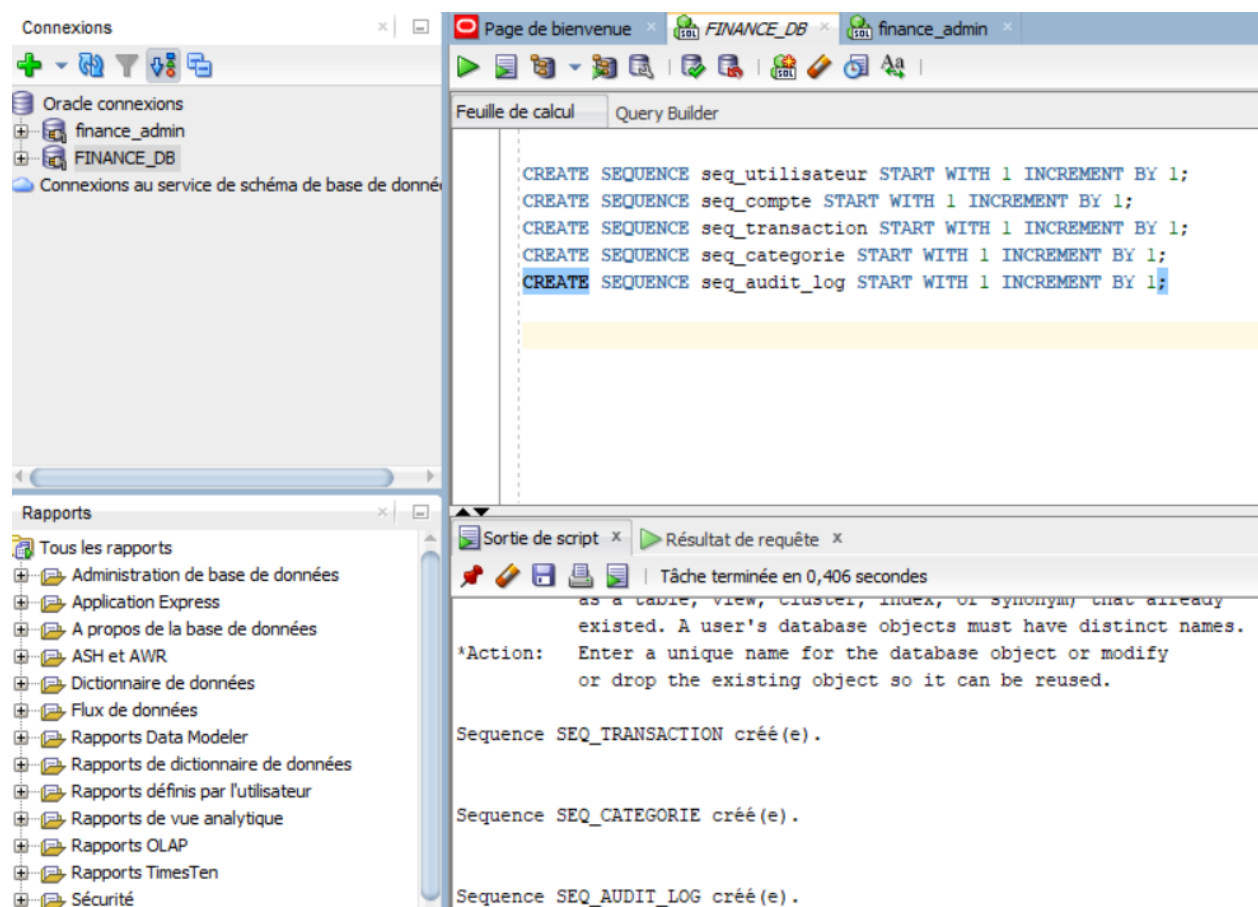
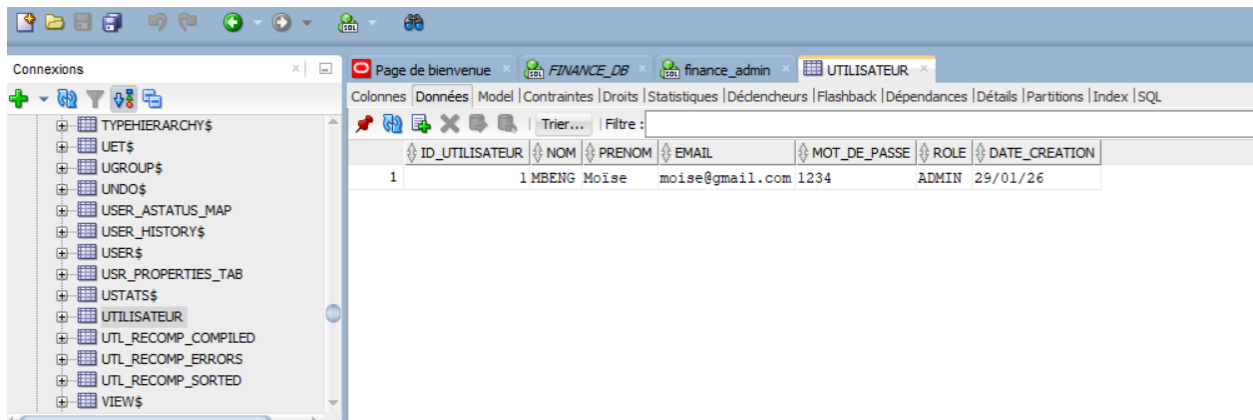


Figure 13:creation des sequence

ÉTAPE 8 : Insertion de données (tests)

Utilisateur

```
INSERT INTO utilisateur VALUES (
    seq_utilisateur.NEXTVAL,
    'MBENG',
    'Moïse',
    'moise@gmail.com',
    '1234',
    'ADMIN',
    SYSDATE
);
```



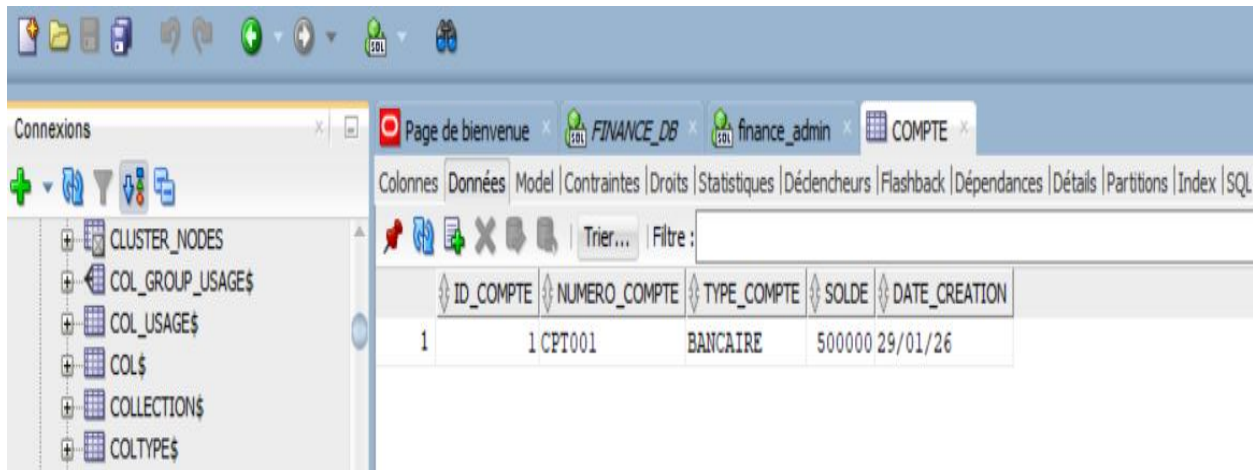
The screenshot shows a database management interface with a tree view on the left and a table view on the right. The table view displays the 'UTILISATEUR' table with the following columns: ID_UTILISATEUR, NOM, PRENOM, EMAIL, MOT_DE_PASSE, ROLE, and DATE_CREATION. A single record is visible with the following values: 1, MBENG, Moïse, moise@gmail.com, 1234, ADMIN, and 29/01/26.

ID_UTILISATEUR	NOM	PRENOM	EMAIL	MOT_DE_PASSE	ROLE	DATE_CREATION
1	MBENG	Moïse	moise@gmail.com	1234	ADMIN	29/01/26

Figure 14:insertion de données

Compte

```
INSERT INTO compte VALUES (
    seq_compte.NEXTVAL,
    'CPT001',
    'BANCAIRE',
    500000,
    SYSDATE
);
```



ID_COMPTES	NUMERO_COMPTES	TYPE_COMPTES	SOLDE	DATE_CREATION
1	1 CPT001	BANCAIRE	500000	29/01/26

Figure 15:insertion compte

Catégorie

INSERT INTO categorie VALUES (

seq_categorie.NEXTVAL,

'Dépenses',

'SORTIE'

);

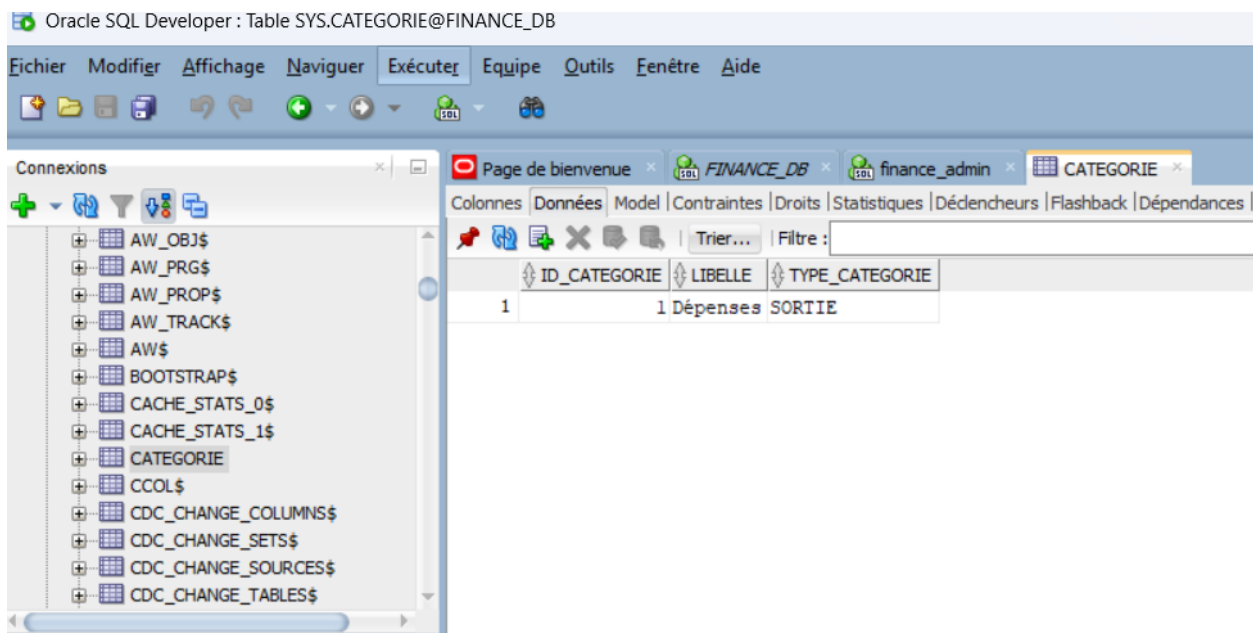


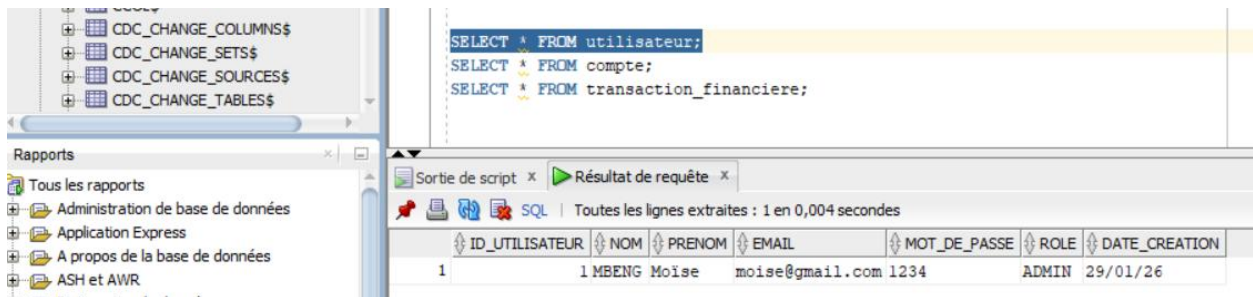
Figure 16:inertion donnees

ÉTAPE 9 : Consultation des données

SELECT * FROM utilisateur;

SELECT * FROM compte;

SELECT * FROM transaction_financiere;

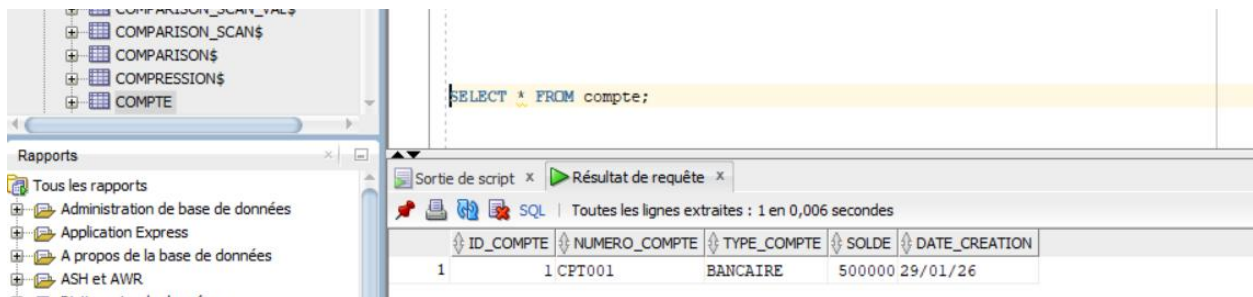


Sortie de script x Résultat de requête x

Toutes les lignes extraites : 1 en 0,004 secondes

ID_UTILISATEUR	NOM	PRENOM	EMAIL	MOT_DE_PASSE	ROLE	DATE_CREATION
1	1 MBENG	Moise	moise@gmail.com	1234	ADMIN	29/01/26

Figure 17: affiche la utilisateur

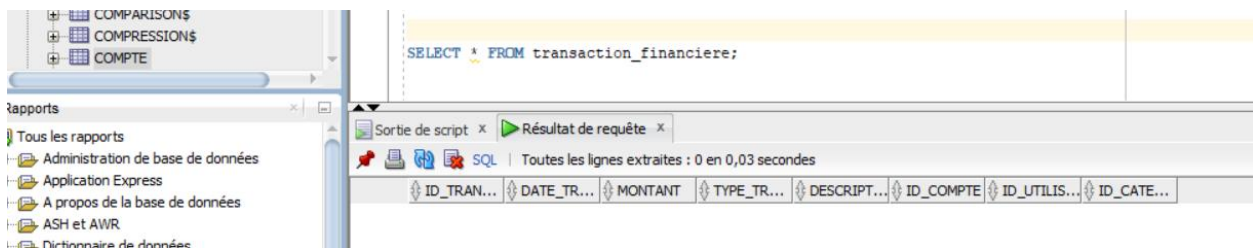


Sortie de script x Résultat de requête x

Toutes les lignes extraites : 1 en 0,006 secondes

ID_COMpte	NUMERO_COMpte	TYPE_COMpte	SOLDE	DATE_CREATION
1	1 CPT001	BANCAIRE	500000	29/01/26

Figure 18: affiche un compte



Sortie de script x Résultat de requête x

Toutes les lignes extraites : 0 en 0,03 secondes

ID_TRAN...	DATE_TR...	MONTANT	TYPE_TR...	DESCRIPT...	ID_COMpte	ID_UTILIS...	ID_CATE...
------------	------------	---------	------------	-------------	-----------	--------------	------------

Figure 19: affiche de la transaction

ÉTAPE 10 : Sécurité et administration (exemple réel)

Audit automatique (exemple manuel)

```
INSERT INTO audit VALUES (
```

```
    seq_audit.NEXTVAL,
```

```
    'Création de transaction',
```

```
    SYSDATE,
```

```
    'finance_admin'
```

```
);
```

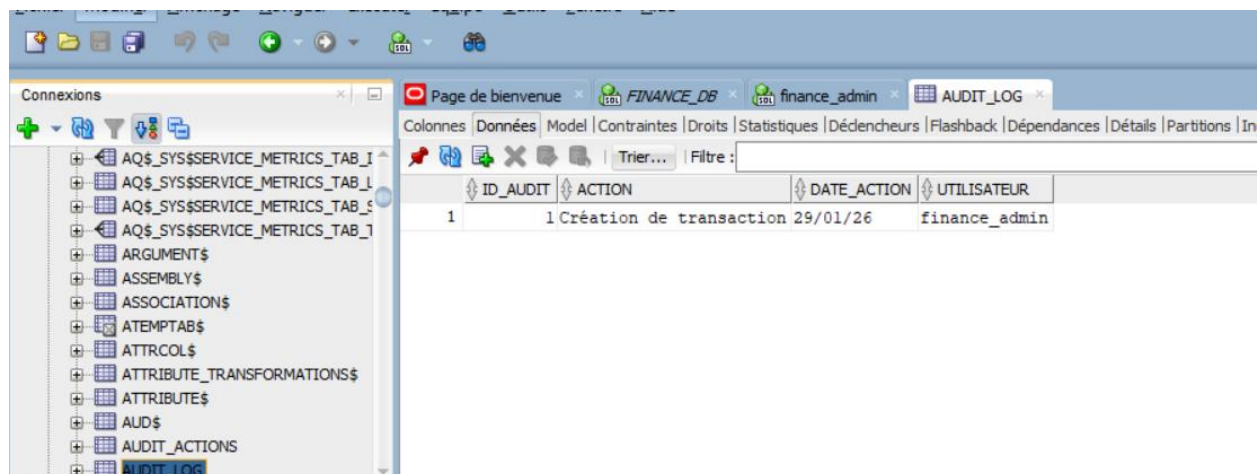


Figure 20: insertion des données de l'audit

ÉTAPE 11 : Sauvegarde (Exportation)

Clic droit sur la connexion

Export Database

Choisir :

Format : SQL

Inclure : Tables, Séquences

Exporter

I. Décomposition

1. **CONSTRAINTfk_compte FOREIGN KEY (id_compte) REFERENCES compte(id_compte)**

- Crée une contrainte appelée fk_compte.
- Déclare que la colonne id_compte de la table courante est une clé étrangère.
- Elle doit obligatoirement correspondre à une valeur existante dans la colonne id_compte de la table compte. Cela garantit que chaque enregistrement fait référence à un compte valide.

2. **CONSTRAINT fk_utilisateur FOREIGN KEY (id_utilisateur) REFERENCES utilisateur(id_utilisateur)**

- Crée une contrainte appelée fk_utilisateur.
- Déclare que la colonne id_utilisateur de la table courante est une clé étrangère.
- Elle doit correspondre à une valeur existante dans la colonne id_utilisateur de la table utilisateur. Cela garantit que chaque enregistrement est lié à un utilisateur existant.

3. **CONSTRAINT fk_categorie FOREIGN KEY (id_categorie) REFERENCES categorie(id_categorie)**

- Crée une contrainte appelée fk_categorie.
- Déclare que la colonne id_categorie de la table courante est une clé étrangère.
- Elle doit correspondre à une valeur existante dans la colonne id_categorie de la table categorie. Cela garantit que chaque enregistrement est associé à une catégorie valide.

II. Utilité en administration & sécurité

- Ces contraintes assurent l'intégrité référentielle : impossible d'insérer une ligne qui pointe vers un compte, un utilisateur ou une catégorie inexistante.
- Elles évitent les incohérences dans la base de données.
- Elles facilitent les jointures entre tables (JOIN) car les relations sont garanties.

Décomposition

1. CREATE TABLE audit → Crée une nouvelle table appelée audit.
2. id_audit NUMBER PRIMARY KEY → Colonne numérique servant d'identifiant unique pour chaque ligne. → La contrainte PRIMARY KEY garantit que chaque valeur est distincte et non nulle.
3. action VARCHAR2(100) → Colonne texte (jusqu'à 100 caractères) qui décrit l'action réalisée (ex. Création de transaction, Suppression de compte).
4. date_action DATE → Colonne de type date qui enregistre le moment exact où l'action a eu lieu.
5. utilisateur VARCHAR2(50) → Colonne texte (jusqu'à 50 caractères) qui indique quel utilisateur a effectué l'action.

Utilité en administration & sécurité

Cette table sert à :

- Tracer les opérations sensibles (création, modification, suppression).
- Identifier les responsables des actions.
- Renforcer la sécurité en gardant un historique des activités.
- Faciliter les audits internes ou externes.

Exemple d'utilisation

Insertion d'un enregistrement dans cette table :

sql

```
INSERT INTO audit VALUES (
    1,
    'Création de transaction',
    SYSDATE,
    'finance_admin');
```

Décomposition :

1. INSERT INTO utilisateur VALUES (...) → Ajoute une nouvelle ligne dans la table utilisateur. La table doit avoir 7 colonnes correspondant aux valeurs fournies.
2. seq_utilisateur.NEXTVAL → Utilise la séquence seq_utilisateur pour générer un identifiant unique automatiquement (comme un compteur qui s'incrémente).
3. 'MBENG' → Nom de famille de l'utilisateur.
4. 'Moïse' → Prénom de l'utilisateur.
5. 'moise@gmail.com' → Adresse e-mail de l'utilisateur.
6. '1234' → Mot de passe de l'utilisateur.
7. 'ADMIN' → Rôle attribué à l'utilisateur (par exemple administrateur).
8. SYSDATE → Fonction Oracle qui insère la date et l'heure système au moment de l'exécution.

Décomposition :

1. INSERT INTO compte VALUES (...) → On insère une nouvelle ligne dans la table compte. La table doit avoir 5 colonnes (par ex. : id_compte, code_compte, type_compte, solde, date_creation).
2. seq_compte.NEXTVAL → Utilise la séquence seq_compte pour générer un identifiant unique automatiquement (comme un compteur qui s'incrémente).
3. 'CPT001' → C'est le code du compte, ici défini manuellement comme CPT001.
4. 'BANCAIRE' → Type du compte, ici indiqué comme « BANCAIRE ».
5. 500000 → Valeur numérique représentant le solde initial du compte (500 000 unités monétaires).
6. SYSDATE → Fonction Oracle qui insère la date et l'heure système au moment de l'exécution, pour tracer la création du compte.

Décomposition :

1. INSERT INTO audit VALUES (...) → On insère une nouvelle ligne dans la table audit. La table audit doit avoir 4 colonnes (par exemple : id_audit, action, date_action, utilisateur).
2. seq_audit.NEXTVAL → C'est une séquence Oracle appelée seq_audit.
 - NEXTVAL signifie « prends la prochaine valeur disponible ».
 - Cela sert à générer automatiquement un identifiant unique pour chaque ligne insérée (comme un compteur qui s'incrémente).
3. 'Création de transaction' → C'est une chaîne de caractères (texte) qui décrit l'action auditée. Ici, on note que l'action est la création d'une transaction.
4. SYSDATE → Fonction Oracle qui renvoie la date et l'heure système actuelles au moment de l'exécution. Cela permet de savoir quand l'action a été réalisée.
5. 'finance_admin' → C'est le nom de l'utilisateur

Processus complet d'exportation d'une base de données

1. Clic droit sur la connexion

Dans ton outil de gestion de base de données (comme Oracle SQL Developer, DBeaver ou un autre IDE), tu as déjà configuré une connexion vers ta base. En effectuant un clic droit sur cette connexion, tu accèdes à un menu contextuel. Ce menu regroupe différentes actions possibles, notamment l'exécution de scripts, l'importation ou l'exportation de données, ainsi que la gestion des objets de la base. C'est le point de départ pour lancer l'exportation.

2. Choisir l'option Export Database

Parmi les options disponibles, tu sélectionnes *Exporter la base de données*. Cette action ouvre un assistant ou une fenêtre dédiée qui te permet de définir les paramètres de l'exportation. L'objectif est de créer une sauvegarde de ta base, soit pour conserver une copie de sécurité, soit pour transférer la structure et les données vers une autre instance.

3. Définir le format : SQL

Tu indiques que le format de sortie doit être un **script SQL**. Ce choix est essentiel car le fichier généré contiendra toutes les instructions nécessaires pour recréer la base ailleurs. On y trouvera :

- Les commandes CREATE TABLE pour reconstruire les tables avec leurs colonnes et contraintes.
- Les commandes CREATE SEQUENCE pour recréer les séquences utilisées comme générateurs d'identifiants.
- Les commandes INSERT INTO pour réinsérer les données existantes.

Ce fichier .sql est donc un véritable mode d'emploi que tu pourras rejouer dans une autre base pour obtenir une copie conforme.

4. Inclure les objets nécessaires : Tables et Séquences

Tu précises que tu veux inclure dans l'export :

- **Les tables** : leur structure (colonnes, types, clés primaires, clés étrangères) et éventuellement leur contenu (les lignes de données).
- **Les séquences** : ces objets Oracle qui servent à générer automatiquement des identifiants uniques.

En incluant ces deux éléments, tu garantis que ta sauvegarde est complète et que tu pourras restaurer non seulement la structure mais aussi le mécanisme d'incrémentation des identifiants.

5. Exporter et générer le fichier

Enfin, tu valides l'opération. L'outil génère un fichier texte au format .sql. Ce fichier est ta **sauvegarde** : tu peux le conserver comme copie de sécurité, le partager avec un collègue, ou le rejouer sur une autre base pour restaurer la structure et les données. En pratique, ce fichier est universel : tant que tu disposes d'un moteur Oracle compatible, tu peux l'exécuter pour recréer la base.

Résultats attendus sur le plan fonctionnel

1 Gestion efficace des utilisateurs

Possibilité d'enregistrer plusieurs utilisateurs.

Attribution de rôles (ADMIN, USER).

Unicité des adresses e-mail.

Une gestion sécurisée et organisée des acteurs du système financier.

2. Gestion des comptes financiers

Création de plusieurs comptes (bancaire, caisse, etc.).

Suivi du solde initial.

Identification unique de chaque compte.

Résultat attendu :

Une représentation fidèle des ressources financières de l'organisation.

2.3 Enregistrement des transactions financières

Insertion correcte des opérations financières.

Distinction entre entrées et sorties.

Historique détaillé des transactions.

Résultat attendu :

Une traçabilité complète des flux financiers permettant l'analyse et le contrôle.

2.4 Classification des opérations par catégorie

Association des transactions à des catégories.

Analyse des dépenses et revenus par type.

Résultat attendu :

Une meilleure visibilité sur la structure financière et les postes de dépenses.

3. Résultats attendus sur le plan sécuritaire

3.1 Traçabilité des actions (Audit)

Enregistrement des actions importantes dans la table AUDIT.

Identification de l'utilisateur ayant effectué l'action.

Horodatage des opérations.

Résultat attendu :

Un système conforme aux exigences de sécurité et de gouvernance des données.

3.2 Réduction des risques de fraude et d'erreur

Contrôle des accès par utilisateur.

Limitation des privilèges.

Possibilité d'analyse a posteriori grâce aux journaux d'audit.

Résultat attendu :

Une base de données plus fiable, plus sûre et plus transparente.

4. Résultats attendus sur le plan pédagogique et académique

4.1 Application concrète des concepts théoriques

Mise en pratique de l'UML (diagramme de classes).

Traduction du modèle conceptuel en modèle physique Oracle.

Application des notions d'administration des bases de données.

Résultat attendu :

Une meilleure maîtrise des concepts fondamentaux des systèmes de gestion de bases de données.

4.2 Respect des normes de développement professionnel

Respect des bonnes pratiques Oracle.

Utilisation correcte des privilèges.

Documentation claire et structurée.

Résultat attendu : Un projet conforme aux standards académiques et professionnels.

5. Résultats attendus globaux du projet

Au terme de ce projet, le système doit : être fonctionnel, être sécurisé, être évolutif, servir de base pour une future application web ou mobile de gestion financière

CONCLUSION

Au terme de ce travail, il apparaît clairement que l'administration d'une base de données de gestion des finances constitue un élément central pour assurer la fiabilité, la sécurité et la performance des systèmes d'information financiers. Dans un contexte où les données financières sont sensibles et stratégiques, leur organisation, leur protection et leur traçabilité deviennent des exigences incontournables. L'étude menée a permis, dans un premier temps, d'analyser l'existant afin de mettre en évidence les limites des systèmes financiers traditionnels, notamment le manque de traçabilité, les risques d'erreurs, l'insuffisance des mécanismes de contrôle et la faible sécurisation des accès. Cette analyse a conduit à la définition d'une problématique claire orientée vers la mise en place d'une solution structurée et fiable. La phase d'analyse et de conception, appuyée sur la modélisation UML, a permis d'identifier les principales entités du système (utilisateur, administrateur, compte, transaction, catégorie, budget, rapport financier et audit) ainsi que les relations qui les lient. Le diagramme de classes élaboré constitue une base solide pour la conception logique et physique de la base de données, garantissant la cohérence des données, la séparation des responsabilités et la traçabilité des opérations financières. La phase de réalisation, basée sur l'utilisation du SGBD Oracle et de l'outil SQL Developer, a permis de mettre en œuvre concrètement l'administration de la base de données à travers la création des tables, la définition des clés primaires et étrangères, la gestion des utilisateurs et des rôles, ainsi que l'intégration des mécanismes de contrôle et d'audit. Ces éléments assurent la sécurité des accès, l'intégrité des données et la responsabilisation des utilisateurs. En définitive, ce projet a démontré que l'administration efficace d'une base de données de gestion des finances repose sur une approche méthodique combinant analyse, conception, sécurisation et contrôle. La solution proposée répond aux exigences fondamentales de gestion financière tout en offrant des perspectives d'évolution, notamment l'automatisation avancée des audits, l'intégration de tableaux de bord décisionnels et le renforcement des mécanismes de sauvegarde et de reprise après incident. Ce travail constitue ainsi une base pertinente pour le développement de systèmes financiers robustes, évolutifs et conformes aux normes de sécurité et de bonne gouvernance des données.

TABLE DES MATIERES

SOMMAIRE.....	1
LISTE DES FIGURES	2
RESUME	3
ABSTRACTE	4
INTRODUCTION GENERALE	5
PARTIE : PHASE TECHNIQUE.....	6
Dossier 1 : L'Existant	7
I. Présentation du thème / projet.....	8
II. Étude de l'existant.....	9
III. Critique de l'existant	10
IV. Problématique	11
V. Proposition de solution.....	12
Dossier 2 : Cahier des charges.....	13
I. Contexte et justification de l'étude / du projet	14
1. Contexte	14
2. Justification	15
II. Les objectifs de l'étude / du projet.....	16
1. Objectif général.....	16
2. Objectifs spécifiques	16
III. Expression des besoins de l'utilisateur	17
1. Besoins fonctionnels	17
2. Besoins non fonctionnels	19
Dossier 3 : analyse	20

I. Méthode et langage d'analyse.....	21
1. Etude comparative UML et MERISE	21
2. Choix et présentation de la démarche d'analyse	23
Dossier 4 : Conception	25
I. Diagramme de classe	26
Dossier 5 : Dossier de Réalisation :	30
I. Présentation de l'environnement logiciel.....	31
1. Environnement logiciel	31
Dossier 6 : Test de fonctionnalités.....	32
I. Fonctionnalités	33
Dossier 7 : Guide d'installation et guide d'utilisateur	36
I. installation d'oracle	37
I. CAS PRATIQUE	45
I. Décomposition.....	64
II. Utilité en administration & sécurité	64
Décomposition	65
Utilité en administration & sécurité	65
Exemple d'utilisation	65
Décomposition :	66
Décomposition :	67
Décomposition :	68
Processus complet d'exportation d'une base de données	69
1. Clic droit sur la connexion	69
2. Choisir l'option Export Database.....	70
3. Définir le format : SQL	71
4. Inclure les objets nécessaires : Tables et Séquences	72
5. Exporter et générer le fichier	73
Résultats attendus sur le plan fonctionnel	74
CONCLUSION	77

TABLE DES MATIERES	78
--------------------------	----